MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

James Gabriel P. Mabagos
Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the
Department of Computer Science
College of Computer Studies, Ateneo de Naga University
in partial fulfillment of the requirements for their respective
Bachelor of Science degrees

Project Advisor: Kevin G. Vega
Estrella H. Montealegre
Michelle Elija B. Santos
Ian Peter L. Lastimosa

September 5 2025
Naga City, Philippines

Keywords: Imperfect Information, DQN, Multi-Agent

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos

Bachelor of Science in Computer Science

Mark Lawrence M. Quibot

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees has been rigorously examined and recommended for approval and acceptance.

Estrella H. Montealegre
Panel Member
Date signed: _____

Michelle Elija B. Santos
Panel Member
Date signed: _____

Ian Peter L. Lastimosa
Panel Member
Date signed: _____

Kevin G. Vega
Project Advisor
Date signed: _____

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos
Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees is hereby approved and accepted by the Department of Computer Science, College of Computer Studies, Ateneo de Naga University.

Lowie Vincent S. Bisana MIT
Chair, Department of Computer Science
Date signed: _____

Joshua C. Martinez MIT
Dean, College of Computer Studies
Date signed: _____

Declaration of Original Work

We declare that the Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

which we submitted to the faculty of the

Department of Computer Science, Ateneo de Naga University

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

James Gabriel P. Mabagos
4 - Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

Kevin G. Vega
Project Advisor

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

by

James Gabriel P. Mabagos and Mark Lawrence M. Quibot

Project Advisor: Kevin G. Vega

Department of Computer Science

ABSTRACT

This research presents MARQ, a Multi-Agent Rainbow Deep Q-Networks, designed to master imperfect information games through the board game Stratego. MARQ leverages a Deep Recurrent Q-Network (DRQN) architecture and league-based self-play training to address the large state space of Stratego and hidden information challenges. The architecture integrates an Attention as Recurrent Neural Network (AAREN) to maintain refined belief states over long game histories, enabling opponent modeling and strategic value contextualization. The system employs Distributional Reinforcement Learning with Noisy Networks to balance strategic unpredictability and stability. The methodology involves developing a game environment and training the AI agent through multi-channel state representations, incorporating Probabilistic Belief States (PBS) and weighted piece values computed through systematic power analysis to optimize decision making under uncertainty. A structured participant study introduces enthusiasts without prior Stratego experience to create a synchronized learning environment, combining quantitative metrics including win rate percentage, average game length, and decision-making speed, with qualitative usability assessments across performance, reliability, and applicability dimensions. Through this study MARQ aims to deliver strategic decisions comparable to human opponents.

ACKNOWLEDGEMENTS

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction

1.1 Project Context

Games have a history of being a metric for how Artificial Intelligence (AI) progresses and performs in the current time [32]. Many real-world problems, from military tactics to multiplayer games, require intelligent decision-making in environments where all information is not available. A common example is a feature found in many multiplayer games known as the "fog of war" [39], where the game environment is not fully revealed, which makes it an example of an imperfect information game [21]. Imperfect information games create a scenario where players develop unconventional strategies without having full knowledge of the game's environmental state [39]. Solving these kinds of problems and determining the optimal strategy remains a significant challenge for Artificial Intelligence [34].

This study focuses on mastering imperfect information games through Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN). In MARL, multiple agents learn and adapt in the same environment, making each decision based on its own observations and rewards [11, 8]. With DQN, the agents can utilize deep neural networks to approximate the Q-function that maps state-action pairs for the expected reward. With all of these together, this study introduces the MARQ (Multi-Agent Rainbow Deep Q-Networks) framework to solve imperfect information games. This approach is suited well for Stratego, where these agents are trained to compete against each other, developing strategies through repeated experience, even though information is lacking about the opponent's setup or the environment's full state.

1.2 Purpose and Description

This research aims to evaluate whether Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) can be used to develop intelligent agents capable of mastering imperfect information games. By focusing on the limited information available to players in environments such as the board game Stratego, this study seeks to address the challenge of making decisions under uncertainty while incorporating strategic planning. Although existing tools for Stratego provide some analytical support, they are often limited in fostering deeper reasoning about the game's tactical complexities.

To advance beyond these limitations, this research seeks to provide an algorithmic foundation for future analytical systems while also emphasizing the development of critical thinking in solving imperfect information games. By modeling adaptive strategies and reasoning under uncertainty, the study highlights how artificial intelligence can mirror and enhance human-like logical reasoning, offering valuable insights for both game analysis and broader decision-making domains.

1.2.1 Research Questions

This study is guided by three main research questions:

1. How effective is the AI agent in adapting to hidden information and different strategies?
2. To what extent does the trained AI agent's performance compare against human players and other AI opponents in Stratego?
3. How does the use of Stratego as a research platform contribute to advancing studies in imperfect information multi-agent reinforcement learning?

1.3 Objectives

The main objective of this study is to develop and evaluate Multi-Agent Reinforcement Learning with Deep Q-Networks for learning appropriate strategies in an imperfect information game, where the agents operate under hidden state variables to improve decision-making, adaptability, and strategic reasoning against both human and AI opponents. This can be achieved through the following:

- Introduction of Stratego to the community

- Design and implement a game environment for Stratego
- Build and implement a training model using Multi-Agent Reinforcement Learning with Deep Q-Networks and AAREN.
- Train and optimize the AI agent to respond to varying game states, hidden information, and opposing strategies.

1.4 Scope and Limitation

The study is limited to the development of an AI agent for imperfect information games, Stratego being the test environment. The scope includes the design and implementation of Stratego as a game environment, the development of a training framework using Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) using AAREN, and the training and optimization of the AI agent to adapt to differences in game states. The evaluation of the agent's performance would primarily be on Stratego's environment. The trainings are limited to the personal devices of the researchers and would not rely on cloud-based computing resources. Since the AI is going to learn alongside the community the player and AI head-to-head will be analyzed mostly qualitative. The goal will be achieving the same human critical thinking in the model.

1.5 Significance of the Study

The development of the MARQ framework addresses limitations in current imperfect information game AI, where existing systems either require large computational resources or employ model-free approaches that limit strategic depth. The MARQ framework's significance lies in its explicit opponent modeling through AAREN and probabilistic belief states, and hybrid DQN architecture that balances strategic depth with computational tractability. The synchronized learning evaluation framework provides insights into AI adaptability against improving human opponents, with implications extending beyond gaming to domains characterized by incomplete information and strategic interaction, including cybersecurity, financial trading, and strategic planning.

1.6 Definition of Terms

AAREN

Attention a recurrent neural network, a model that integrates the sequential processing of RNNs with the parallel efficiency of Transformers, designed for tasks such as event forecasting and classification.

Alpha-Beta Pruning

A search algorithm that explores possible moves in a game tree while pruning branches that cannot affect the final decision, making the search process more efficient.

Counterfactual Regret Minimization (CFR)

An iterative algorithm that approximates Nash equilibrium by minimizing regret, widely applied in imperfect information games such as Poker and Stratego.

Deep Q-Networks (DQN)

A reinforcement learning algorithm that combines Q-learning with deep neural networks, enabling agents to approximate optimal decision-making in complex environments.

Fog of War

A game mechanic used in strategy games that conceals parts of the map or opponent actions, requiring players to utilize prediction and information-gathering strategies to reduce uncertainty.

Imperfect Information Games

Games where players have incomplete knowledge of the current game state, requiring them to make strategic decisions under uncertainty (e.g., Stratego, Poker, Multiplayer Online Battle Arenas).

Knowledge-Limited Unfrozen Subgame Solving(KLUSS)

Knowledge-Limited Unfrozen Subgame Solving (KLUSS) is an advancement in search algorithms for imperfect-information games. KLUSS builds on the earlier Knowledge-Limited Subgame Solving (KLSS) method, which simplified computation by removing irrelevant states and freezing certain

strategies.

Long Short-Term Memory (LSTM)

A variant of RNN designed to capture long-term dependencies in sequential data while mitigating the vanishing gradient problem.

Multi-Agent Reinforcement Learning (MARL)

An extension of reinforcement learning in which multiple agents interact within the same environment, learning coordinated or competitive strategies through simultaneous training.

Nash Equilibrium

A game-theoretic concept where no player can gain an advantage by unilaterally changing their strategy, often used as a benchmark for decision-making in imperfect information games.

Perfect Information Games

Games in which all players have complete knowledge of the current game state and available moves, such as Chess and Go.

Proximal Policy Optimization (PPO)

A reinforcement learning algorithm that optimizes policies through probability ratio clipping, often compared to DQN in performance within gaming environments such as VizDoom.

Recurrent Neural Network (RNN)

A type of neural network designed to process sequential data by maintaining memory of previous inputs, useful in imperfect information games for recalling past moves or revealed information.

Reinforcement Learning (RL)

A machine learning paradigm where agents learn optimal strategies through trial-and-error interactions with the environment, guided by rewards and penalties.

Stratego

A two-player strategy board game characterized by hidden information and asymmetric piece abilities, where the objective is to capture the opponent's flag or render them immobile.

Student of Games (SoG)

A unified AI framework combining self-play learning, strategic search, and game-theoretic reasoning, designed to handle both perfect and imperfect information games.

Vanishing Gradient

The vanishing gradient problem is basically as in the name, vanishing gradient, as color in a gradient, the message in this case starts strong and gradually fades away when it passes through each layer, making the learning weak or close to zero.

Chapter 2

Review of Related Literature

2.1 Artificial Intelligence in Imperfect Information Games

Games can be classified as an imperfect information game by having incomplete information about the current game state for both players [32, 11]. An example would be in MOBAs (Multiplayer Online Battle Arena), the fog of war mechanic hides portions of the map, including objectives and enemy positions, from both teams. By placing a tool, such as a ward in League of Legends or an observer in Dota 2, the player gains a temporary vision in the map for strategic planning [9]. This feature engages players to create strategic reasoning under certainty, utilizing prediction, coordination, and resource management to gain informational advantages over opponents. Games have a history of being a metric on how AI progresses and performance in the current time [32, 15]. The case is the same for both perfect information and imperfect information games [32]. For perfect information games, like chess, they primarily use Alpha-Beta pruning, a search algorithm that uses trees to explore possible chess moves and prunes the branches that will not affect the final decision [26]. In imperfect information games, a search algorithm like Alpha-Beta Pruning would be difficult to implement because the algorithm is made for perfect information games [31]. A popular algorithm that is used in imperfect information games would be Monte Carlo Tree Search with Reinforcement Learning because MCTS has integration with neural network systems like AlphaZero [27]. In terms of reinforcement learning, the training focuses on trial and error to provide the best possible decision [42]. Continuing development of AI algorithms in imperfect information games

advances our understanding of decision-making under uncertainty and promotes strategic planning.

2.1.1 Student of Games

A unified learning algorithm that can support both perfect information and imperfect information is Student of Games. Combining self-play learning, strategic search, and principles from game theory [32]. SoG achieved strong performance in Chess and Go in perfect information games and in imperfect information games like heads-up no-limit Texas hold 'em poker, and defeated the state-of-the-art agent in Scotland Yard. When comparing to MARL, SoG supports and has been tested in both game types and uses game theoretic reasoning, which computes or approximates the Nash equilibria. MARL with DQN uses trial and error learning and self-play learning, which relies on the exploration of the environment. MARL has also been tested in Starcraft II, which is a real-time strategy video game, unlike SoG, which has not been tested in video games [18].

2.1.2 Deep Q-Networks

One of the most important deep-learning algorithms is called Deep Q-Networks [16]. This algorithm is a type of reinforcement learning algorithm that combines Q-Learning with deep neural networks [6]. A study has been made comparing DQN with Proximal Policy Optimization(PPO) in VizDoom, a 3D environment for reinforcement learning based on Doom, a first-person-based shooter [37]. In the study, the environment would be in a gamemode called DeathMatch, where the main goal is to kill as many bots while minimizing deaths. In PPO with object detection, it learned a policy to run around and collect survival items like medkits and armor rather than focusing on kills, making the bot have close to zero kills. While the bot using DQN with object detection would go for both kills and collecting survival items, making the kill-death ratio higher than PPO. In short, PPO with object detection focuses on survival by collecting survival items and avoiding combat, while DQN with object detection has a balance of collecting survival items and combat.

2.1.3 Multi-Agent Reinforcement Learning

Reinforcement Learning can solve complex problems through trial and error, by learning from the environment to be able to provide optimal decisions [11]. This can be the case for Single-Agent Reinforcement Learning(SARL). SARL has its limits in solving many real-world problems. Multi-

Agent Reinforcement Learning(MARL) was introduced as a solution for the challenges of SARL, by having multiple agents in an environment that interact with each other, the decisions they could make would be optimized [11, 8]. Optimized, meaning that these agents go through trial and error together to make the best decision given the reward. As for the implementation of this in Stratego, two agents would compete with each other to know the optimal decisions given the environment, which has imperfect information and opposing strategies. As MARL has its advantages, it also has challenges. The first would be Non-Stationarity, and the second is scalability, as for non-stationarity, each agent learns simultaneously, meaning the environment constantly changes [8]. For scalability, as more agents are being used, the computational power rises, making it expensive [23, 8].

2.1.4 KLUSS

Knowledge-Limited Unfrozen Subgame Solving (KLUSS) was introduced as an advancement in search algorithms for imperfect-information games.[39] KLUSS builds on the earlier Knowledge-Limited Subgame Solving (KLSS) method, which simplified computation by removing irrelevant states and freezing certain strategies. Unlike KLSS, KLUSS "unfreezes" these strategies, allowing them to be re-optimized during subgame solving. This flexibility improves strategic reasoning, such as bluffing and adaptation, and was a key factor in achieving high-level performance in Fog of War chess.

2.1.5 Counterfactual Regret Minimization

Counterfactual Regret Minimization(CFR) is an iterative algorithm that approximates the Nash equilibria commonly used on imperfect information games like poker and Stratego [23, 36]. The Nash equilibria are what CFR approximates to make the best possible decision with minimal regret. Regret in this scenario is the what-ifs of the algorithm if the algorithm chooses the other decision. For CFR to make decisions, it repeatedly minimizes regret to approximate the Nash equilibria [23]. Other works have made variations of the base CFR, showing improved convergence rate, but require a significant amount of effort by researchers. A framework is proposed named AutoCFR, instead of relying on manually crafted algorithms, it automatically designs new CFR variants using an outer loop to search over algorithm designs and an inner loop to test them on equilibrium finding tasks [35].

2.2 RNN

Recurrent Neural Network is a type of Neural Network designed to process sequential data. Unlike common Neural Networks, RNNs have a memory because the information passed to the next step is based on the memory from the previous step. RNN can be applied to imperfect information games such as Stratego because of its memory. By using the memory to remember the pieces that have been revealed, the decision could be optimized [27].

2.2.1 LSTM

Long Short-Term Memory is a popular RNN variant that has the capabilities of RNN but also processes data with long-term dependencies [2]. LSTM mitigates the vanishing gradient problem that RNN faces. The vanishing gradient problem is basically as in the name, vanishing gradient, as color in a gradient, the message in this case starts strong and gradually fades away when it passes through each layer, making the learning weak or close to zero. LSTM can be used to solve imperfect information games because of its capability to process data with long-term dependencies while having the memory of RNN.

2.2.2 Aaren

Aaren, known as Attention as a recurrent neural network, combines the parallel training efficiency of Transformers with the streaming update efficiency of RNNs [13]. Aaren, like transformers, can be trained in parallel as well as RNN, can process sequential data with memory. In the study, Aaren was tested in event forecasting, time series forecasting, and classifications. Aaren achieved similar accuracy to Transformers but was more time and memory-efficient.

2.3 Stratego

The origin of Stratego can be traced to a traditional board game called Jungle, where instead of human pieces instead it used animals. In the jungle, the pieces are not hidden, making it a perfect information game. Stratego is a two-player board game where players each have a 40-piece army. In the tabletop version of Stratego, one player is assigned to the blue army and the other player is assigned to the red army. Each army has a flag assigned to it, which cannot be moved when

the game starts. There are a total of 80 troop pieces, while the total number of variations is 12. The pieces rank range from 1 to 10, and a piece that's not numbered is a bomb and the flag. The player also has time to move pieces before the game starts to make use of the fog of war and create creativity and optimal positions on the board. As the pieces have ranks, the higher the number, the higher the power. There is an instance where the lower-ranked can defeat a higher-ranked ranked, which is the piece that has a rank of one, named the spy, can defeat the strongest piece that has a rank of ten, named the marshal, if the spy attacks first. As for the bombs, almost all pieces are vulnerable except the miner, the only one who can defuse the bomb. And the pieces that cannot be moved are the bomb and the flag. Another piece that has a special ability is the scout; it can move either horizontally or vertically, not just one square. If both pieces have the same number, and the piece attacks, both of them would be eliminated. There are multiple win conditions in Stratego. The obvious one would be capturing the flag, another one would be that the opponents have no mobile pieces, another would be that there are no miners that can defuse the bomb to capture the flag, and lastly, the opposing player had the time run out.

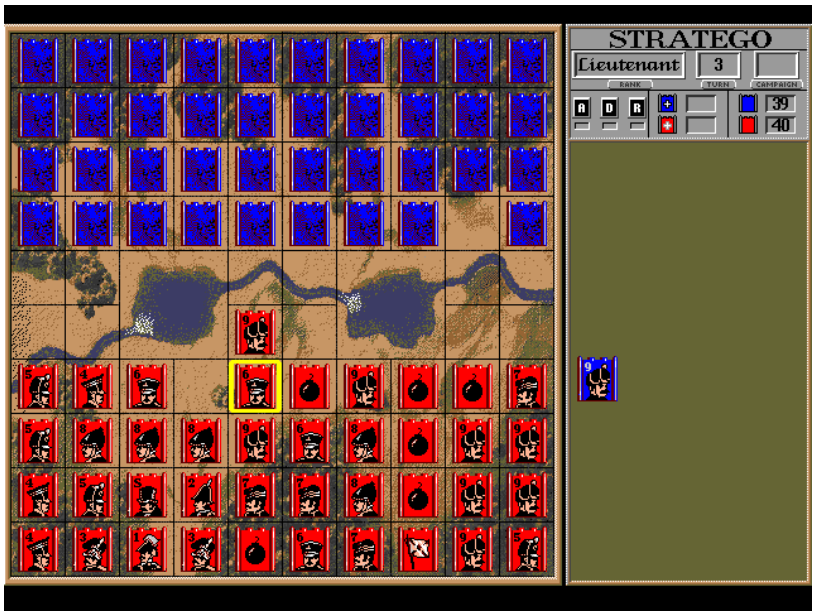

Figure 2.1: Stratego Game board.

2.4 Board Games in Community Building

Modern board games are becoming more widespread, expanding their market share each year. During COVID-19 lockdowns, board game sales increased because people were stuck at home with family and had free time. Also, playing board games during that time helped with stress, isolation, and anxiety. Certain games like Pandemic, a cooperative strategy board game where players work together to stop global disease outbreaks by discovering cures before time runs out, became more popular due to their reflection of the real-world scenario. Board game communities differ between the West and the East [14]. From the east, board games are considered social activities, while in the west, board games grew as hobbies, communities focusing on the gameplay and mechanics. In Hong Kong, there are two distinct board game communities named BGHK and Oasis. BGHK consists of Cantonese-speaking individuals who treat board games as social bonding. They even event sessions for "party game days" and "welcome newbies," emphasizing making friends rather than the game itself. While the Oasis group focuses on the hobby, playing newly released and complex games. Attracting players with the mechanics and novelty rather than just socializing. A study has been made to use board games as an answer to the problem of difficulty in creating a sense of "togetherness or kinship," and board games have advantages for improving the cognitive function of older people [3]. In the study, the Tresna Werdha Budi Pertiwi Social Home has a problem of having a sense of family among residents. By playing together, the elderly had consistent interaction with each other, reducing the feeling of loneliness and detachment. Board games not only provide entertainment but also memory stimulation and improve social bonds. Board games provide a face-to-face interaction requiring people to sit together at a table, unlike in video games, where players are often separate from each other [29]. Every shared experience creates a unique microenvironment where players enter a separate reality governed by the game's rules. In the space, the players share emotions and strategy. Sharing these emotions helps strengthen friendships, family bonds, and trust. Many board game players first connect through gaming with friends and family, which can branch out to dedicated groups or conventions, and expand their social circle.

2.5 Other Applications

An extension of MARL with DQN was demonstrated in robotics. In these environments, robots are trained to make decisions based on rewards. The study assigned different energy costs to each

robotic arm, allowing the robot to learn how to adjust its movement based on cost, using less energy when the reward is small and exerting more effort if the reward is appropriate [25]. MARL with Deep Q-learning was also used in traffic signal control. Applied a cooperative multi-agent DQN framework to manage six interconnected intersections, where each intersection acted as an independent learning agent. Using the SUMO simulator, each agent selected signal phases to minimize congestion and improve overall traffic flow. Their approach introduced a reward function based on the standard deviation of stopping vehicles across lanes, showing balanced lane utilization rather than simply reducing queue lengths. This design showed that MARL with DQN can enhance not only traffic efficiency but also environmental sustainability by reducing emissions and fuel consumption. In businesses in developing countries, it is essential that growth should be the top priority; however, this creates a problem in increasing taxation for improving infrastructure to further support the current population. Using MARL, we can create a model system where multi-agent RL acts as different parties that discover the optimal tax policies without needing to rely on traditional economic models [41].

2.6 Application to Other Game Environments

Although this study focuses on Stratego as the game environment, the MARQ framework can also be applied to other strategic and simulation-based games that involve decision-making and hidden information. Sports management titles such as Madden NFL's Dynasty Mode, NBA 2K's franchise mode, Esports Manager, and Football Manager feature complex systems of player development, transfer, and long-term planning, where agents must act without complete information about the future player performance and market behavior. In these environments, the MARQ framework could model AI agents capable of learning optimal trade strategies or budget allocations through reinforcement learning. The framework's belief-state modeling would allow the AI to check hidden player attributes or opponent strategies, while the Deep Q-Network could optimize sequential decisions across multiple simulated seasons.

Similarly, the framework could be applied to a game called Clash Royale, where players must make fast tactical decisions without full knowledge of their opponent's hand or resource level. The Probabilistic Belief State in this setting could estimate the likelihood of specific opponent cards being available, while the Deep Q-Network would select optimal actions for troop deployment and

timing. The KLUSS component of the framework could divide the match into smaller subgames, such as lane control or counter-attack scenarios.

Chapter 3

Theoretical Frameworks

The main idea of the MARQ framework, a Multi-Agent Rainbow Deep Q-Networks, is to employ self-play techniques and deep reinforcement learning to train agents capable of handling Stratego's large game state space and imperfect information. The MARQ framework uses a league-based self-play system where agents learn by competing against a diverse pool of opponents, including their own previous versions and exploratory variants.

This approach addresses the fundamental challenge of Stratego: reasoning about hidden information without computing intractable common-knowledge sets. While the number of possible game states exceeds 10^{535} and initial setups alone reach 10^{66} per player [28], the MARQ framework manages this complexity through belief state management, probabilistic reasoning, and strategic pattern recognition. The framework combines deep reinforcement learning for value estimation, attention mechanisms for processing game history, and controlled strategy mixing to prevent exploitation, while maintaining computational tractability even in this vast imperfect-information domain.

3.1 MARQ Architecture Integration

The MARQ framework operates through the hierarchical integration of its components. The system starts the observation processing, where the AAREN module ingests raw game positions to construct and maintain a probabilistic belief state(PBS). This belief state forms the base of information used for all subsequent reasoning.

Finally, the entire architecture is embedded within a league-based training paradigm. This

involves the simultaneous training and evaluation of MARQ agents.

3.1.1 Convergence and Theoretical Guarantees

The league training system ensures improvement through three complementary mechanisms. First, the system maintains best response values against historical opponents, ensuring that the current policy remains competitive against the full distribution of strategies encountered during training. Second, promotion to the main agent pool requires a positive win rate against previous versions before replacement, preventing regression in overall performance. Third, diversity bonuses in the opponent selection distribution prevent premature convergence to strategies that may be exploitable by novel counter-strategies.

These properties, combined with extensive validation, provide confidence that the MARQ framework will achieve strong, robust play despite the theoretical compromises necessary for tractability.

3.2 MARQ Policy Definition

Policy optimization in reinforcement learning refers to the iterative refinement of an agent's decision-making strategy to maximize cumulative expected reward. In multi-agent environments characterized by imperfect information, this optimization process must produce policies that are not only effective against static opponents but also robust to the inherent uncertainty of hidden game states and the adaptive behaviors of learning adversaries. The MARQ framework addresses these challenges through a hierarchical policy architecture that integrates belief state reasoning with value-based decision making.

The policy learning process in MARQ operates across multiple interconnected objectives. The primary objective concerns game outcomes, where the agent learns to maximize win probability through strategic piece deployment and tactical combat decisions. A secondary objective involves belief state accuracy, where the Probabilistic Belief State maintained by the AAREN module is continuously refined through comparison with revealed ground truth positions during gameplay. These objectives are jointly optimized, as accurate belief states enable better decision-making, while strategic decisions that reveal opponent information improve future belief quality.

Table 3.1: Key Symbols and Descriptions from the MARQ Framework

Symbol	**Description**	**Example (MARQ Framework)**
$\pi_i(a\|I_i)$	Policy for agent i choosing action a given information set I_i	Probability of moving a piece forward given current visible board state and belief about opponent pieces
$\mathcal{B}_t$	Belief state at time t representing probability distributions over opponent configurations	Probability distribution over possible identities and positions of hidden opponent pieces
$v_i^{\pi}(h, \mathcal{B}_t)$	Value function for agent i following policy π given history h and belief state $\mathcal{B}_t$	Expected reward for current board position considering uncertainty about opponent setup
$Q_{\text{network}}(s', a)$	Deep Q-Network approximation of action-value function	Neural network estimate of value for attacking with a suspected Marshal vs opponent piece
$R^{\text{capture}}i, t+1$	Immediate reward from piece captures weighted by piece values	+9 points for capturing opponent Marshal, -1 for losing a Scout
$O = \{o_1, \ldots, o_t\}$	Observation sequence processed by AAREN attention mechanism	Sequence of board states, moves, and attack outcomes throughout the game
α_i	Attention weights for historical observations in AAREN	Higher weight on turn when opponent's Marshal was revealed, lower on routine Scout moves
π^*	Nash equilibrium policy profile	Optimal mixed strategy that cannot be exploited by opponent strategy changes
$Q_{eval}(\mathcal{B})$	PBS evaluator network output (quality score)	Confidence score for belief distribution accuracy
$H(\pi)$	Policy entropy measure	Ensures minimum unpredictability to prevent exploitation
ϵ	Exploration rate for agent behavior	Controls exploration vs exploitation balance in multi-agent learning
γ	Discount factor for future rewards	Importance of long-term vs immediate rewards in value estimation

3.2.1 Core Policy Framework

A policy in the MARQ framework, denoted as $\pi_i(a|I_i)$, defines the probability distribution over actions for agent i conditioned on its current information set I_i. Unlike policies in perfect information games that operate on complete game states, MARQ policies must reason under uncertainty about the true state of the environment. The information set I_i contains all observations available to the agent, including visible board positions, revealed piece identities, and the history of observed movements and combat outcomes.

The fundamental challenge in imperfect information games lies in the requirement that agents make decisions without access to the opponent's private information. In Stratego, this private information includes the identity and position of all opponent pieces that have not yet been revealed through combat. The policy must therefore integrate uncertain beliefs about opponent configurations into its decision process, selecting actions that perform well across the range of possible hidden states.

To address this challenge, the MARQ policy operates on belief states $\mathcal{B}_t$ rather than attempting to enumerate all consistent game states. The belief state represents a probability distribution over possible opponent piece configurations, maintained and updated by the AAREN module as new observations become available. The policy is thus defined as:

$$\pi_i(a|I_i, \mathcal{B}_t) = P(\text{action} = a \mid \text{information set } I_i, \text{belief state } \mathcal{B}_t) \tag{3.1}$$

This formulation captures the key insight that optimal play in imperfect information games requires probabilistic reasoning over hidden states. The policy learns to weight actions based on their expected value across the belief distribution, rather than optimizing for any single assumed opponent configuration.

PBS-to-Reality Optimization

The quality of the Probabilistic Belief State directly impacts decision quality, as inaccurate beliefs lead to poorly calibrated action values. MARQ therefore incorporates belief accuracy as an auxiliary training objective, using revealed piece identities as supervised labels for belief refinement.

When combat reveals the true identity of opponent pieces, the prediction error between the prior belief and the revealed ground truth is measured using Kullback-Leibler divergence:

$$\mathcal{E}_{\text{pred}}(t) = \sum_{p \in \text{Pieces}_{\text{revealed}}(t)} D_{\text{KL}}\left(\text{PBS}_t(p) \| \delta_{s_p^*}\right) \tag{3.2}$$

Here, $\delta_{s_p^*}$ denotes the Dirac delta distribution concentrated on the true piece type s_p^* at the moment of revelation. This error quantifies how surprised the belief system was by the revealed information, with larger values indicating greater divergence between predicted and actual piece identities.

This prediction error is transformed into a belief accuracy reward signal that encourages the policy to develop beliefs that better predict actual piece configurations:

$$R_{\text{accuracy}}(t) = -\mathcal{E}_{\text{pred}}(t) = -\sum_{p \in \text{Pieces}_{\text{revealed}}(t)} \left[\text{PBS}_t(p = s_p^*) > 0\right] \cdot \log \text{PBS}_t(p = s_p^*) \tag{3.3}$$

The optimal policy is then defined as the policy that maximizes the joint objective of game outcomes and belief accuracy:

$$\pi_i^* = \arg\max_{\pi_i} \mathbb{E}_{\tau \sim \pi_i}\left[\sum_{t=0}^{T} \gamma^t \left(R_{\text{game}}(t) + \eta \cdot R_{\text{accuracy}}(t)\right)\right] \tag{3.4}$$

The hyperparameter $\eta \in \mathbb{R}^+$ controls the relative importance of belief accuracy versus immediate game rewards. Higher values of η produce policies that prioritize information gathering and belief refinement, while lower values emphasize direct strategic objectives. The optimal balance depends on game phase, with information gathering more valuable in early and middle game positions where uncertainty remains high.

Belief Calibration Loss. Beyond individual prediction accuracy, the MARQ framework monitors systematic biases in belief predictions through a calibration loss computed across training episodes:

$$\mathcal{L}_{\text{calib}} = \frac{1}{|\mathcal{D}|} \sum_{(t,p) \in \mathcal{D}} \left\| \text{PBS}_t(p) - \mathbb{1}[s_p = s_p^*] \right\|_2^2 \tag{3.5}$$

The dataset $\mathcal{D}$ aggregates all revelation events across training, and $\mathbb{1}[s_p = s_p^*]$ is the indicator vector with value one at the true piece type and zero elsewhere. This loss identifies cases where the belief system systematically over-predicts or under-predicts certain piece types, enabling targeted corrections to the belief update mechanism.

3.2.2 Bellman Equation with Belief States

The value function in MARQ extends the standard Bellman equation to incorporate belief state evolution. The recursive value decomposition expresses the expected cumulative reward as a function of the current information set and belief state:

$$v_i^\pi(I, \mathcal{B}_t) = \mathbb{E}_\pi \left[R_{i,t+1}^{\text{capture}} + \lambda_1 R_{i,t+1}^{\text{info}} + \lambda_2 R_{i,t+1}^{\text{position}} + \gamma v_i^\pi(I_{t+1}, \mathcal{B}_{t+1}) \mid I_t = I, \mathcal{B}_t \right] \tag{3.6}$$

The reward function decomposes into three components that capture different aspects of strategic value. The capture reward R^{capture} reflects the immediate material consequences of combat, weighted by the relative values of pieces captured and lost. The information reward R^{info} quantifies the strategic value of revealing opponent pieces, as this information improves future decision quality. The positional reward R^{position} measures territorial control and piece mobility, encouraging positions that provide strategic advantages. The discount factor γ determines the relative weight placed on immediate versus future rewards, and $\mathcal{B}_{t+1}$ represents the updated belief state after observing action outcomes.

3.2.3 Nash Equilibrium in Imperfect Information

The strategic objective of MARQ training is to discover policies that approximate Nash equilibrium strategies. A Nash equilibrium represents a strategy profile where no player can improve their expected payoff through unilateral deviation. In the context of imperfect information games, the equilibrium condition is expressed as:

$$v_i(\pi^*) = \max_{\pi_i} v_i(\pi_i, \pi_{-i}^*) \tag{3.7}$$

This condition states that the equilibrium policy π^* for player i maximizes expected value against the equilibrium strategies of all other players π_{-i}^*. Finding exact Nash equilibria in large imperfect information games is computationally intractable, so MARQ instead seeks approximate equilibria through self-play training with diverse opponent populations.

3.2.4 Implementation Framework

The practical implementation of the MARQ policy relies on an attention-based memory system that processes observation sequences to maintain coherent belief representations. Given an observation

sequence $O = \{o_1, \ldots, o_t\}$ representing the history of visible board states and action outcomes, the AAREN module computes attention weights that determine the relevance of each historical observation to the current belief state:

$$\alpha_i = \text{softmax}(W_q h_t \cdot W_k o_i) \tag{3.8}$$

These attention weights enable the system to selectively aggregate information from the game history, prioritizing observations that are most informative for current decision-making. The resulting belief representation is then used by the Rainbow DQN to estimate action values and select moves.

The multi-agent learning process maintains several agent populations that interact during training. The main agents represent the current best-performing policies and are actively optimized through gradient descent. Exploration variants with modified action selection mechanisms discover novel strategic approaches that may otherwise remain unexplored. Historical checkpoints of previous agent versions prevent strategy forgetting by ensuring that new policies maintain competence against earlier strategies.

3.2.5 Convergence Properties

The MARQ training procedure provides probabilistic guarantees on convergence to approximate equilibrium strategies. For any tolerance level $\epsilon > 0$, the average strategy profile $(\bar{x}, \bar{y})$ computed over training iterations converges to an ϵ-Nash equilibrium of the constructed game as the number of training iterations T approaches infinity.

The league training methodology ensures monotonic improvement toward equilibrium through several mechanisms. The system maintains best response values against historical opponents, ensuring that new policies perform at least as well as their predecessors against the training distribution. Promotion to the main agent pool requires a positive win rate against the previous best policy, preventing regression in overall performance. Diversity bonuses in the opponent selection distribution prevent premature convergence to strategies that may be exploitable by novel counter-strategies.

These theoretical guarantees, combined with empirical validation across extensive training runs, establish that the MARQ framework can discover effective and robust strategies for imperfect information gameplay while remaining computationally tractable on consumer-grade hardware.

3.3 Game Environment and State Management

The Stratego environment provides the foundation for agent training and evaluation, managing the complex dynamics of imperfect information and simultaneous move selection.

3.3.1 Information Set Management

The environment maintains distinct representations for different information contexts:

$$\mathcal{I}_{player} = \{s : s \text{ is consistent with player observations}\} \tag{3.9}$$

Each player receives only the subset of game state information available through their observation function.

3.3.2 State Space Complexity

The theoretical state space of Stratego demonstrates the challenge addressed by MARQ:

$$|\mathcal{S}| = \binom{40}{1, 1, 2, 3, 4, 4, 4, 5, 8} \times 10^{10} \times 2^{40} \approx 10^{535} \tag{3.10}$$

This high complexity necessitates knowledge-limited reasoning and belief state management.

3.3.3 Temporal State Evolution

Game states evolve according to the update function:

$$s_{t+1} = \text{EnvStep}(s_t, a_t^1, a_t^2, \theta_t) \tag{3.11}$$

where θ_t represents stochastic elements (hidden information revelation, time limits, etc.).

3.3.4 Valid Action Filtering

The environment enforces game rules through valid action constraints:

$$\mathcal{A}_{valid}(s, player) = \{a : \text{IsLegalMove}(a, s, player) \wedge \text{RespectsGameRules}(a, s)\} \tag{3.12}$$

This ensures that agents can only select legal moves consistent with Stratego's rules and piece capabilities.

3.3.5 Piece Setup and Initialization

The initial game setup involves strategic piece placement governed by setup policies:

$$\pi_{setup}(\text{placement}|player) = \text{NeuralNetworkSetup}(\text{strategic\_features}, \theta_{setup}) \tag{3.13}$$

where strategic features include positional value maps, piece interaction potentials, and defensive vulnerabilities.

The setup phase represents a critical strategic decision that influences the entire game trajectory, making it a key component of the overall MARQ training framework [10].

3.4 Theoretical Convergence Guarantees

The MARQ framework provides theoretical guarantees for convergence and performance under specific conditions.

3.4.1 Incremental Learning Bounds

Under standard RL assumptions (independent identical distribution of experiences, bounded rewards), the MARQ algorithm provides the following bound on value function estimation:

$$\|V_t - V^*\|_\infty \leq \frac{C}{\sqrt{t}} + \epsilon_{approx} \tag{3.14}$$

where C depends on reward bounds and discount factor, and ϵ_{approx} represents function approximation error.

3.4.2 Belief State Accuracy Bounds

The PBS accuracy improves with additional observations according to:

$$\mathbb{E}[\text{Accuracy}(\mathcal{B}_t)] \geq 1 - \exp(-\alpha \cdot N_{obs}(t)) \tag{3.15}$$

where $N_{obs}(t)$ is the cumulative number of informative observations and α is a positive constant.

3.4.3 Strategy Convergence in Self-Play

The league training methodology ensures convergence to ϵ-Nash equilibria under standard monotonic improvement assumptions:

$$\lim_{T \to \infty} \|\pi_T - \pi^*\| \leq \epsilon \tag{3.16}$$

for some $\epsilon > 0$ depending on the exploration parameters and opponent diversity.

These theoretical foundations provide confidence in the MARQ framework's ability to learn robust, effective strategies in the challenging domain of imperfect information strategic gameplay.

3.5 Deep Q-Network with Belief States

Deep Q-Networks in the MARQ framework operate on belief states rather than complete game states. The network approximates Q-values for probability distributions over opponent configurations, $\mathcal{B}_t$:

$$Q(s, a) = \mathbb{E}_{s' \sim \mathcal{S}_t \subseteq \mathcal{B}_t}[Q_{network}(s', a)]$$

The system uses a **Standard Replay Buffer** for experience storage, sampling transitions $(\mathcal{B}_t, a_t, r_t, \mathcal{B}_{t+1})$ uniformly to train the value function.

3.5.1 Rainbow DQN Components

The implementation incorporates key enhancements from the Rainbow DQN architecture to improve stability and sample efficiency:

Distributional RL (Categorical DQN)

Stratego's inherent uncertainty leads to multimodal value distributions. We model the value distribution $Z(s, a)$ directly rather than just its expectation. The distribution is approximated by a discrete distribution over $N = 51$ atoms $\{z_i\}_{i=1}^{N}$, with support ranging from V_{min} to V_{max}:

$$Z_\theta(s, a) = \sum_{i=1}^{N} p_i(s, a)\delta_{z_i} \tag{3.17}$$

The training objective minimizes the Kullback-Leibler divergence between the predicted distribution and the projected target distribution:

$$D_{KL}(\Phi\hat{\mathcal{T}}Z_{\theta'}||Z_\theta) \tag{3.18}$$

where Φ is the projection operator mapping the Bellman update onto the fixed support atoms.

Noisy Networks for Exploration

To ensure consistent exploration without the need for discrete ϵ-greedy schedules, we employ Noisy Linear layers. The weights and biases are perturbed by a deterministic function of factorized Gaussian noise:

$$y = (\mu_w + \sigma_w \odot \epsilon_w)x + (\mu_b + \sigma_b \odot \epsilon_b) \tag{3.19}$$

where ϵ_w, ϵ_b are random variables. This allows the network to learn the degree of exploration required for different states.

Dueling Network Architecture

The network explicitly separates the estimation of state values $V(s)$ and advantage values $A(s, a)$ using two separate streams that share a common convolutional feature extractor:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right) \tag{3.20}$$

This decomposition leads to better policy evaluation in the presence of many similar-valued actions.

3.5.2 Exploration-Driven Q-Value Adjustment

We augment the Q-values with an exploration bonus based on the uncertainty of the belief state, utilizing an "optimism in the face of uncertainty" approach:

$$Q_{augmented}(s, a) = Q_{base}(s, a) + \lambda \cdot U(a) \tag{3.21}$$

where $U(a)$ represents the entropy of the belief distribution at the target location, encouraging the agent to investigate unknown opponent pieces.

3.5.3 Adaptive Learning Rate Scheduling

MARQ employs adaptive learning rate adjustment based on loss trends and performance metrics. The learning rate η_t is dynamically adjusted according to:

$$\eta_{t+1} = \begin{cases} \eta_t \cdot \alpha_{decay} & \text{if } \mathcal{L}_{recent} > \mathcal{L}_{baseline} \cdot \tau_{increase} \\ \eta_t \cdot \alpha_{increase} & \text{if } \mathcal{L}_{recent} < \eta_{threshold} \\ \eta_t \cdot \alpha_{time} & \text{time-based decay} \end{cases} \tag{3.22}$$

where $\mathcal{L}_{recent}$ is the average loss over recent training steps, $\tau_{increase}$ is the increase threshold, and α parameters control the rate of adjustment.

3.5.4 Cross-System Performance Monitoring

The framework includes comprehensive performance monitoring to detect misalignment between PBS and DQN components. The system tracks four key metrics: PBS prediction accuracy trends that measure how well the belief state predicts revealed piece identities, DQN loss convergence patterns that indicate training stability, action prediction alignment scores that compare selected actions against high-value alternatives, and reward distribution analysis that monitors the statistical properties of received rewards across episodes.

Misalignment is detected when PBS accuracy falls below $\theta_{pbs} = 0.5$ while DQN loss exceeds $\theta_{dqn} = 0.5$, triggering corrective measures in the training process.

3.6 Attention as Recurrent Neural Network (AAREN)

The AAREN module acts as the core memory and belief state system within the MARQ architecture. It is specifically designed to process extended sequences of partial observations in Stratego, integrating information over hundreds of moves while avoiding the vanishing gradient problem that affects recurrent models like LSTMs or GRUs in long tasks. AAREN leverages an attention mechanism to provide direct, weighted access to the entire history of observations, enabling more coherent and computationally efficient belief tracking [13].

Recurrent Attention Computation. AAREN implements an attention-based recurrent computation that maintains a state triplet (a_t, c_t, m_t) across the observation sequence. Unlike traditional recurrent neural networks that suffer from vanishing gradients over long sequences, AAREN provides direct access to the entire history through attention weights while maintaining O(1) inference complexity per timestep.

Given a sequence of observations $O = \{o_1, \ldots, o_t\}$ where each o_i encapsulates the visible board state and public action outcomes, the AAREN cell first projects each observation into key and value representations:

$$k_t = W_k(o_t), \quad v_t = W_v(o_t) \tag{3.23}$$

The projection matrices W_k and W_v are learned during training. The key k_t determines how relevant the current observation is for belief updates, while the value v_t encodes the informational content to be integrated into the belief representation. The attention score s_t measures the relevance of the current observation relative to the learned query vector q:

$$s_t = q^T k_t \tag{3.24}$$

Higher attention scores indicate observations that are more informative for piece identification. To prevent numerical overflow when computing exponentials of attention scores, AAREN maintains a running maximum $m_t = \max(m_{t-1}, s_t)$, enabling the log-sum-exp trick for numerical stability even for sequences spanning hundreds of moves.

The weighted value accumulator a_t aggregates value representations across the sequence, with weights determined by the attention scores:

$$a_t = a_{t-1} \exp(m_{t-1} - m_t) + v_t \exp(s_t - m_t) \tag{3.25}$$

The term $\exp(m_{t-1} - m_t)$ rescales the previous accumulator when a new maximum is encountered, while $\exp(s_t - m_t)$ computes the relative attention weight for the current observation. The normalization constant $c_t = c_{t-1} \exp(m_{t-1} - m_t) + \exp(s_t - m_t)$ ensures that the effective attention weights sum to unity. Finally, the output representation $h_t = a_t / c_t$ represents an attention-weighted aggregation of all observations up to time t, with more informative observations receiving higher weights.

Parallel Training and Piece Value Inference. For training efficiency, AAREN employs parallel prefix scan computation across the observation sequence. This allows processing of entire sequences in parallel while maintaining the O(1) per-timestep inference complexity for deployment. The parallel implementation computes cumulative sums of exponentially-weighted values, enabling efficient GPU utilization during training.

The AAREN model learns to infer piece values from action sequences using 24-dimensional feature vectors encoding movement patterns, contextual information, and game state features. The network outputs probability distributions over piece types:

$$P(\text{piece\_type} = t \mid \text{action\_sequence}) = \text{softmax}(W_{out} \cdot h_T + b_{out})_t \tag{3.26}$$

This allows the PBS to combine AAREN predictions with rule-based inference and historical pattern matching for robust piece identification under uncertainty.

3.7 PBS Quality Evaluation Framework

The MARQ framework incorporates a PBS evaluation system that assesses prediction quality and provides feedback for continuous improvement. This system addresses the challenge of determining when and how much to trust PBS predictions.

Evaluator Network and Quality Assessment. The PBS Evaluator employs a neural network architecture that takes belief distributions as input and outputs quality scores $Q_{eval}(\mathcal{B}) = \text{MLP}(\mathcal{B}; \theta_{eval})$, where $\mathcal{B}$ is the belief distribution over piece types. The evaluator learns to predict the expected reward of PBS predictions based on historical outcomes. The training objective is based on rewards computed from ground truth comparisons:

$$R_{quality}(\mathcal{B}, g) = \alpha \cdot \mathcal{B}[g] - \beta \cdot |V_{pred} - V_{actual}| - \gamma \cdot \text{distance\_penalty} \tag{3.27}$$

In this formulation, g represents the ground truth piece type revealed during combat, while $V_{pred} = \sum_t r_t \cdot \mathcal{B}[t]$ is the expected piece value computed from the belief distribution and V_{actual} is the actual piece rank. Higher-value pieces receive greater reward or penalty weighting, reflecting their strategic importance in the game.

Bias Detection and Feature Learning. The system includes bias tracking that identifies systematic over/under-prediction patterns for different piece types. For each piece type t, a correction factor $\phi_t = \frac{\text{actual\_rate}_t}{\text{predicted\_rate}_t} \cdot \text{accuracy\_adjustment}_t$ is computed, and corrected belief distributions are obtained as $\mathcal{B}_{corrected}[t] = \phi_t \cdot \mathcal{B}[t]$.

The framework also learns which action features are most predictive for PBS inference through a dedicated neural network that outputs importance weights $\mathbf{w}_{importance} = \text{FeatureNet}(\mathbf{x}_{action}; \theta_{feat})$ for the 24-dimensional action feature vector.

Active Learning for Information Gathering. The system determines when additional information gathering is beneficial through uncertainty assessment. A position requires more observation when both the entropy of beliefs exceeds a threshold and the quality score falls below a quality threshold:

$$\text{NeedMoreInfo}(\mathcal{B}) = \mathbb{1}\{H(\mathcal{B}) > \theta_H\} \cdot \mathbb{1}\{Q_{eval}(\mathcal{B}) < \theta_Q\} \tag{3.28}$$

This active learning approach prioritizes scouting moves that can reduce uncertainty in strategically important positions.

3.8 Specialized Agent Architectures

3.8.1 Setup Agent via Distributional RL

The game's initial setup is handled by a specialized SetupAgent that learns optimal piece configurations by treating the placement phase as a sequential decision process. The state representation consists of a 3-channel input tensor encoding the current board state with already-placed pieces, a valid positions mask indicating legal placement locations, and a current piece type mask identifying which piece is being placed.

The agent outputs a distributional Q-value for each of the 100 board positions, mapping $Q(s, pos) \rightarrow$ Distribution over $[V_{min}, V_{max}]$. This distributional approach captures the inherent uncertainty in setup quality, as the true value of a placement depends on the unknown opponent configuration.

To prevent creating static, easily countered setups, the agent includes an exploitability critic network that penalizes predictable placements. The total training loss combines the C51 distributional loss with a predictability penalty:

$$\mathcal{L}_{total} = \mathcal{L}_{C51} + \lambda \cdot \mathcal{L}_{predictability} \tag{3.29}$$

3.8.2 Information Set Monte Carlo Tree Search (IS-MCTS)

For high-stakes decision making, particularly in the endgame, MARQ employs an Information Set MCTS agent that combines search with learned value functions. The search process proceeds in three phases.

First, during the determinization phase, the agent samples a consistent concrete world state s_{det} from the PBS beliefs according to $s_{det} \sim P(S|\mathcal{B}_t)$. This converts the imperfect information game into a perfect information instance for the duration of the simulation.

Second, during tree traversal, the search tree is navigated using a UCB1 variant to balance exploration and exploitation of move sequences. Nodes in the tree represent information sets rather than individual states, allowing the tree to be shared across multiple determinizations.

Third, during leaf evaluation, terminal or unexpanded nodes are evaluated using the Rainbow DQN value function rather than random rollouts, providing a high-quality heuristic estimate:

$$V(s_{leaf}) \approx Q_{DRQN}(s_{leaf}, a_{best}) \tag{3.30}$$

This hybrid approach allows deep lookahead while respecting the uncertainty inherent in the belief state.

3.9 Multi-Agent Reinforcement Learning

MARQ employs a league reinforcement learning system that minimizes non-stationarity, a fundamental challenge in multi-agent learning where policy updates by one agent alter the environment experienced by others. The system mitigates this through several mechanisms:

3.9.1 League-Based Training Architecture

The league consists of a dynamic pool of agents designed to ensure robust generalizability. The main agent represents the current best-performing policy that is being actively optimized through gradient descent. The historical pool contains a saved collection of past main agent checkpoints representing distinct strategic eras discovered during training.

During training, opponents are sampled using a 50/50 distribution:

$$P(\text{opponent}) = \begin{cases} 0.5 & \text{Main Agent (Self-Play)} \\ 0.5 & \text{Uniform(Historical Pool)} \end{cases} \tag{3.31}$$

This prevents "cycling" where the agent re-learns and un-learns the same strategies endlessly, forcing it to defeat all previous versions of itself.

3.9.2 Experience Replay

The framework employs a **Standard Replay Buffer** that stores transitions $(\mathcal{B}_t, a_t, r_t, \mathcal{B}_{t+1}, \pi_t)$. Uniform sampling is used to stabilize the training distribution across the agent's history.

3.9.3 Multi-Dimensional Reward Structure

The reward function $R(s, a, s')$ is carefully shaped to guide learning in the sparse-reward Stratego environment:

$$R_{total} = R_{move} + R_{capture} + R_{tactical} + R_{info} \tag{3.32}$$

The move reward R_{move} provides small rewards for advancing pieces (approximately +0.05) to encourage active play and prevent overly defensive strategies. The capture reward $R_{capture}$ is scaled by the rank of the captured piece according to $0.15 \times \frac{\text{Rank}}{11.0}$, providing proportional feedback for material gains. The tactical reward $R_{tactical}$ gives specific bonuses for key strategic captures, including +0.5 for Miner vs Bomb defusals and +1.0 for Spy vs Marshal assassinations. The information reward R_{info} provides +0.3 for revealing high-value opponent pieces with Rank ≥ 9. Game phase multipliers adjust these weights dynamically, prioritizing material accumulation in the early game and safe positioning in the endgame.

3.10 Strategy Mixing

Extensive-form imperfect information games with deterministic strategies are inherently suboptimal and vulnerable to exploitation. An adversary with knowledge of a deterministic policy effectively reduces the game to a perfect information setting, nullifying the strategic uncertainty fundamental

to games like Stratego. The MARQ framework addresses this challenge through two complementary mechanisms that provide stochasticity at different levels of the decision hierarchy.

Action-Level Stochasticity via Noisy Networks. At the action selection level, MARQ employs Noisy Linear layers that inject learned, state-dependent noise into the policy. Rather than using fixed exploration schedules such as ϵ-greedy, the network learns the appropriate degree of randomization for each game state. The noisy layer output is computed as:

$$y = (\mu_w + \sigma_w \odot \epsilon_w)x + (\mu_b + \sigma_b \odot \epsilon_b) \tag{3.33}$$

where μ_w, μ_b are learned mean parameters, σ_w, σ_b are learned noise scale parameters, and ϵ_w, ϵ_b are factorized Gaussian noise samples regenerated at each forward pass. This approach provides exploitation resistance without explicit entropy constraints, as the learned noise ensures that even an adversary with access to the network weights cannot perfectly predict action selection.

Strategy-Level Diversity via League Training. At the strategy level, MARQ achieves mixing through diverse opponent exposure during training. The league system maintains a population of historical agent checkpoints representing different strategic approaches discovered during training. The opponent selection distribution ensures that the main agent must perform well against a mixture of strategies rather than optimizing against a single fixed opponent, including historical league agents (50%), random agents (20%), greedy heuristics (20%), and self-play (10%). The combination of action-level noise and strategy-level diversity produces policies that are both unpredictable in individual games and robust across different opponent types.

3.11 Stratego Game Domain

Stratego is an imperfect information board game with European roots, popularized in the Netherlands in the 1940s and released in North America in 1961. The game is played on a 10x10 board that includes two 2x2 "lake" areas in the center, which are impassable. Each player commands an army of 40 pieces of varying ranks, and during the game, the identity and rank of the pieces are kept secret from the opposing player. The objective is to capture the opponent's flag or eliminate all of their movable pieces.

Each army consists of 12 different types of pieces, with ranks ranging from 1 (the Spy) to 10 (the Marshal), alongside Bombs and a Flag. Higher-ranked pieces defeat lower-ranked ones during combat, except in special situations: the Spy can defeat a Marshal if attacking first, the Miner is the only piece that can defuse Bombs, and Scouts can move multiple squares in a straight line. Victory is achieved by capturing the opponent's Flag, immobilizing all movable enemy pieces, or forcing the opponent to run out of time.

Figure 3.1: Stratego Pieces

3.12 League Training Implementation

The league training system maintains a collection of historical agent checkpoints, enabling training against diverse opponents that represent different strategic approaches discovered during training. This prevents policy cycling and ensures generalization.

3.12.1 League Manager Architecture

The `LeagueManager` class persists agent checkpoints as PyTorch state dictionaries, storing both the network weights and episode metadata. The league capacity is capped at $N_{max} = 50$ agents;

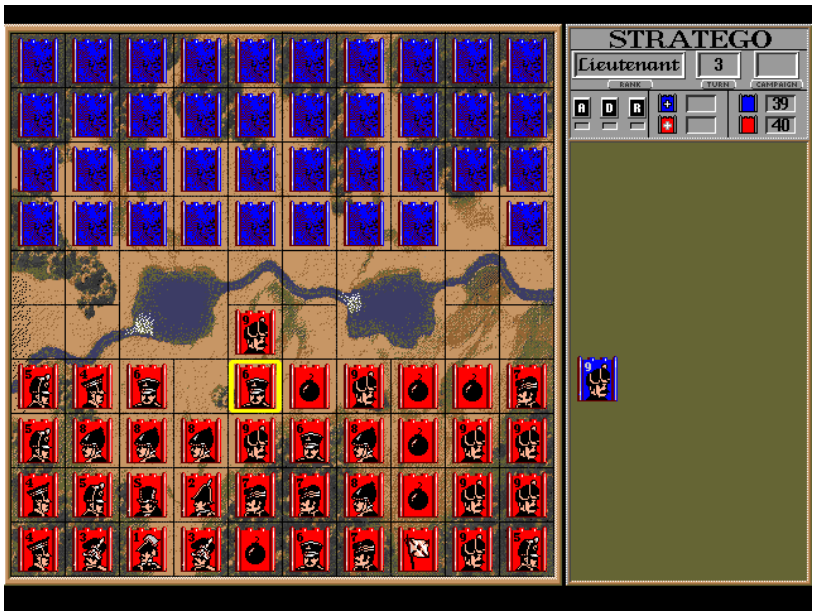

Figure 3.2: Stratego Game board.

when this limit is exceeded, the oldest checkpoint is removed. New checkpoints are saved every 500 episodes, ensuring that distinct strategic eras are captured without excessive storage overhead.

Opponent selection follows a configurable probability distribution during training:

$$P(\text{opponent}) = \begin{cases} 0.5 & \text{League (historical agents)} \\ 0.2 & \text{Random agent} \\ 0.2 & \text{Greedy heuristic} \\ 0.1 & \text{Self-play (current weights)} \end{cases} \tag{3.34}$$

3.12.2 Opponent Pool Dynamics

The `OpponentPool` class implements the opponent selection logic. At each episode, a uniform random value $r \in [0, 1)$ determines which opponent type is selected. If the league is non-empty and $r < P_{league}$, an agent is sampled uniformly from the historical pool. Otherwise, the random agent, greedy heuristic, or current self are selected based on the cumulative probability thresholds.

$$\text{select\_opponent}() = \begin{cases} \text{Uniform}(\mathcal{L}) & \text{if } |\mathcal{L}| > 0 \wedge r < P_{league} \\ \pi_{random} & \text{if } r < P_{league} + P_{random} \\ \pi_{greedy} & \text{if } r < P_{league} + P_{random} + P_{greedy} \\ \pi_{self} & \text{otherwise} \end{cases} \tag{3.35}$$

3.12.3 Exploiter Agents

Three specialized exploiter agents probe specific weaknesses. The **RusherAgent** prioritizes aggressive forward advancement, adding +0.5 to Q-values for each row moved toward the opponent and +1.0 for any attack action. The **TurtleAgent** implements a defensive stance with a −2.0 penalty for attacks and +0.3 bonus for remaining in the player's starting territory (rows 6-9 for Player 1). The **FlankingAgent** focuses on the board edges, adding +0.4 for moves in columns 0-2 or 7-9, while penalizing central columns 3-6 by −0.2. These exploiters are introduced during Phase 4 to verify the main agent handles diverse playstyles.

3.13 Reward Shaping Module

The `RewardCalculator` class computes multi-component rewards with configurable weights. The default configuration assigns weight 1.0 to game outcomes, 0.5 to material exchanges, 0.3 to information gain, and 0.2 to positional advantage.

3.13.1 Material Reward

Combat outcomes generate material rewards scaled by piece rank. When the attacking piece wins, it receives a base reward of +0.5 plus an additional $0.05 \times \frac{\text{captured\_rank}}{11}$, proportional to the captured piece's value. Losing an attack incurs −0.3 minus a rank-scaled penalty. Mutual destruction results in zero material reward.

$$R_{material} = \begin{cases} +0.5 + 0.05 \times \frac{\text{captured\_rank}}{11} & \text{if attacker wins} \\ -0.3 - 0.05 \times \frac{\text{lost\_rank}}{11} & \text{if defender wins} \\ 0 & \text{if mutual destruction} \end{cases} \tag{3.36}$$

Special cases receive fixed bonuses: Miners defusing Bombs earn $R_{special} = +0.5$, Spies assassinating the Marshal earn +1.0, and capturing the Flag yields the terminal reward $R_{terminal} = +10.0$.

3.13.2 Epistemic Reward

Information gain is quantified as the reduction in belief entropy when pieces are revealed:

$$R_{epistemic} = \sum_{pos \in \text{revealed}} (H_{before}(\mathcal{B}_{pos}) - H_{after}(\mathcal{B}_{pos})) \tag{3.37}$$

where $H(\mathcal{B}) = -\sum_t \mathcal{B}[t] \log \mathcal{B}[t]$ is the Shannon entropy of the belief distribution. Additional small rewards are given for Scout identification via multi-square movement (+0.02), Bomb discovery through failed attacks (+0.03), and narrowing the probable Flag location (+0.05).

3.13.3 Positional Reward (PBRS)

Potential-based reward shaping (PBRS) provides dense feedback while preserving optimal policy invariance. The reward is computed as the discounted difference in state potential:

$$R_{positional} = \gamma \Phi(s') - \Phi(s) \tag{3.38}$$

The potential function $\Phi(s)$ combines three terms: the negative Manhattan distance from friendly pieces to the enemy Flag region (encouraging advancement), a bonus of +0.01 per piece occupying central columns 3-6 (rewarding board control), and +0.05 for Miners positioned near suspected Bomb locations (facilitating defusal):

$$\Phi(s) = w_1 \cdot \text{FlagDistance}(s) + w_2 \cdot \text{CenterControl}(s) + w_3 \cdot \text{MinerPosition}(s) \tag{3.39}$$

3.14 Information Hiding Architecture

The environment maintains separate board tensors to enforce Stratego's hidden information constraints.

3.14.1 Board Representation

Three tensors represent the game state: `actual_board` contains the ground truth with all piece values, while `visible_board_p1` and `visible_board_p2` store each player's observed state. Enemy pieces appear as a constant `HIDDEN_PIECE = -20` in visible boards, encoding the presence of an opponent piece without revealing its rank.

3.14.2 Observation Function

The game state returned to agents depends on the training mode. During curriculum Phase 1, full observability is enabled and agents receive the actual board. In all other phases, agents receive their player-specific visible board:

$$\mathcal{O}_{player}(s) = \begin{cases} s_{actual} & \text{if full\_observability} \\ s_{visible}^{player} & \text{otherwise} \end{cases} \tag{3.40}$$

Each cell in the visible board is set according to:

$$s_{visible}^{player}[r, c] = \begin{cases} s_{actual}[r, c] & \text{if own piece or previously revealed} \\ \text{HIDDEN} & \text{if unrevealed enemy piece} \\ \text{EMPTY} & \text{if empty square} \\ \text{LAKE} & \text{if lake square} \end{cases} \tag{3.41}$$

3.14.3 Piece Revelation

When a battle occurs, the `reveal_pieces` function updates both visible boards simultaneously:

$$\text{reveal\_pieces}(pos_1, pos_2) \rightarrow \begin{cases} s_{visible}^{p1}[pos_1] \leftarrow s_{actual}[pos_1] \\ s_{visible}^{p1}[pos_2] \leftarrow s_{actual}[pos_2] \\ s_{visible}^{p2}[pos_1] \leftarrow s_{actual}[pos_1] \\ s_{visible}^{p2}[pos_2] \leftarrow s_{actual}[pos_2] \end{cases} \tag{3.42}$$

Revelations are permanent and tracked for PBS updates.

3.15 PBS Update on Piece Revelation

When a battle reveals a piece's true type, the PBS synchronizes with ground truth through several update steps.

3.15.1 Belief Collapse

Upon revealing piece type g at position pos, the belief distribution collapses to certainty:

$$\mathcal{B}_{pos}[t] = \begin{cases} 1.0 & \text{if } t = g \\ 0.0 & \text{otherwise} \end{cases} \tag{3.43}$$

3.15.2 Constraint Propagation

The revealed piece count is incremented, and remaining beliefs are adjusted to respect piece count constraints. For unrevealed positions $p \neq pos$:

$$\mathcal{B}_p[t] \leftarrow \mathcal{B}_p[t] \cdot \frac{\text{max\_count}[t] - \text{revealed\_count}[t]}{\text{remaining\_count}[t]} \tag{3.44}$$

For example, when the single Marshal is revealed, the probability of any other piece being a Marshal drops to zero.

3.15.3 Training Data Collection

Each revelation generates a supervised training example for AAREN. The action sequence observed for the now-revealed piece becomes a labeled sample:

$$\mathcal{D}_{AAREN} \leftarrow \mathcal{D}_{AAREN} \cup \{(\text{action\_sequence}_{pos}, g, pos)\} \tag{3.45}$$

3.16 Curriculum Learning

Training is organized into five phases that progressively introduce complexity, starting with simplified conditions and advancing to full league training.

3.16.1 Phase 1: Full Observability

The agent initially trains with `full_observability = True`, allowing it to see all piece values. This removes uncertainty and enables learning of basic game mechanics: piece movement rules, combat outcomes, and tactical patterns. Opponents consist of random agents (60%) and heuristic policies (40%). Training continues for 500 to 2000 episodes until the agent achieves $\geq 90\%$ win rate against random and $\geq 60\%$ against heuristic.

3.16.2 Phase 2: Partial Observability

Observability is disabled (`full_observability = False`), requiring the agent to rely on the PBS for hidden pieces. The opponent is a frozen heuristic agent, providing a stable training target. This phase runs for 1000 to 3000 episodes, with transition requiring PBS accuracy $\geq 70\%$ and win rate $\geq 55\%$.

3.16.3 Phase 3: Self-Play

The agent trains against a delayed copy of itself (from 500 episodes prior), discovering novel strategies through iterated self-improvement. This phase runs 2000 to 5000 episodes, transitioning when the win rate variance falls below 0.1, indicating strategic stability.

3.16.4 Phase 4: League Training

The full opponent distribution is activated: league agents (50%), random (20%), greedy heuristic (20%), and self-play (10%). The exploiter agents (Rusher, Turtle, Flanking) are periodically introduced to probe for weaknesses. This is the main training phase, running for 5000+ episodes.

3.16.5 Phase 5: Scenario Drills

Every 1000 episodes during Phase 4, specialized board configurations are initialized for targeted training: Miner vs Bomb defusal patterns, Marshal tracking and elimination, and endgame Flag search. These drills reinforce specific tactical skills that may be underrepresented in normal games.

The phase transition logic follows:

$$\text{Transition}(\textit{phase}) = (\text{episodes} \geq \min) \wedge (\text{metrics} \geq \text{thresholds}) \tag{3.46}$$

3.17 Reward Shaping

The reward function is carefully designed with four components to guide learning in the sparse-reward Stratego environment.

Material Rewards. Combat outcomes are weighted by piece strategic value according to $R_{material} = w_{capture} \cdot \text{CaptureReward} + w_{loss} \cdot \text{LossPenalty} + w_{special} \cdot \text{SpecialBonus}$. Special bonuses are awarded for tactically significant captures: a Miner defusing a Bomb receives +0.5 for enabling strategic flag access, a Spy assassinating the Marshal receives +1.0 reflecting the game-changing nature of eliminating the highest-ranked piece, and favorable trades receive $+0.1 \times (V_{captured} - V_{lost})$ based on the value differential.

Epistemic Rewards. Information gain is measured through entropy reduction: $R_{epistemic} = \alpha \cdot \Delta H(\mathcal{B}) + \beta \cdot \text{ScoutDeduction} + \gamma \cdot \text{BombIdentification}$, where $\Delta H(\mathcal{B}) = H(\mathcal{B}_{t-1}) - H(\mathcal{B}_t)$ represents the reduction in belief entropy. Scout identification provides immediate entropy reduction when multi-square movement is observed.

Positional Rewards. Territorial control is rewarded through potential-based reward shaping (PBRS): $R_{positional} = \gamma\Phi(s') - \Phi(s)$. The potential function $\Phi(s)$ combines flag proximity, center control, and Miner positioning near suspected Bombs. PBRS guarantees that the optimal policy remains unchanged while providing denser learning signals.

Composite Reward. Auxiliary prediction tasks provide additional learning signals through $R_{prediction} = \lambda_{piece} \cdot \text{PieceTypeAccuracy} + \lambda_{outcome} \cdot \text{BattleOutcomePrediction}$. The total reward combines all components with configurable weights:

$$R_{total} = w_o \cdot R_{outcome} + w_m \cdot R_{material} + w_e \cdot R_{epistemic} + w_p \cdot R_{positional} \tag{3.47}$$

Default weights are: $w_o = 1.0$, $w_m = 0.5$, $w_e = 0.3$, $w_p = 0.2$.

3.18 Parallel Environment Architecture

MARQ utilizes parallel environment execution to accelerate training through batch processing. Each environment runs in a separate process to avoid Python GIL limitations, with workers communicating

via pipes to receive commands and return game states. The parallel environment supports dynamic observability switching for curriculum learning, allowing seamless transition between training phases.

Actions, states, and PBS updates are processed in batches: $\text{step\_batch}(\mathbf{a}) \rightarrow (\mathbf{s}', \mathbf{r}, \mathbf{d}, \mathbf{info})$, where vectors represent parallel environment states, enabling efficient GPU utilization.

Chapter 4

Methodology

This chapter outlines the methodology employed to evaluate the effectiveness of an AI system based on the MARQ framework in solving mind sports with imperfect information. The approach involves the development of a competitive system, followed by the evaluation of its performance through win-rate analysis against human players and an assessment of its adaptability across diverse game environments. The subsequent sections detail the processes and methods that will be used to achieve these research objectives.

4.1 Project Planning

This research will be conducted according to a structured plan encompassing participant selection, sample size determination, and experimental procedures. The project will follow a rapid prototyping approach for the design and implementation of the AI system. The timeline is segmented into four distinct phases. The development phase will run from September 2025 to February 2026, beginning with game environment development from September through November 2025, followed by AI training from December 2025 to January 2026, and concluding with system deployment in February 2026. The user testing phase will take place in March 2026, involving one-on-one matches and data collection activities. The final analysis and documentation phase will commence in April 2026, focusing on data analysis and manuscript preparation.

4.1.1 Community Introduction to Stratego

The introduction of the game Stratego to the participant community is a prerequisite for this study. This necessity arises from a complete lack of available local participants possessing prior knowledge of the game's rules or strategies. Unlike more popular games like Chess, Stratego's niche status means a pre-existing pool of experienced players is not available for participation to test against the MARQ framework. Therefore, a structured introduction is essential to cultivate the necessary participant cohort from the ground up. This approach ensures all participants begin with a uniform baseline understanding. It allows for the parallel development of both human expertise and artificial intelligence, creating a synchronized learning environment where the adaptive capabilities of the MARQ framework can be fairly assessed against a human cohort that is also in the process of learning.

4.1.2 Participant Selection

The participant selection process is designed to recruit a cohort of 20 individuals via convenience sampling, taking into account the project's resource constraints while still yielding a enough sample. The target demographic is enthusiasts of strategic mind sports of a diverse age and game experience.

Prospective participants will be screened to evaluate two key attributes. Their strategic game experience is key to ensuring a baseline aptitude, and second, their confirmed lack of prior knowledge of Stratego to ensure the study's "synchronized learning" premise. Ultimately, only those who successfully finish the screening phase will be admitted, guaranteeing they possess the necessary foundational skills to provide meaningful interaction with the MARQ framework and substantive feedback.

4.1.3 Experimental Procedures

One-on-one matches between human participants and the MARQ framework will be conducted on-site using a single device. To ensure consistency, each participant will receive a checklist outlining the procedures for interacting with the game.

Each session will last one (1) hour, during which participants must complete at least two (2) games against the MARQ framework. Additional games may be played if the time permits. The following game statistics will be recorded will include the winner, total game duration, total moves

per player, and the number and type of remaining pieces for both players. Immediately following the session, each participant will be given an evaluation form to provide open-ended feedback on their experience.

4.2 System Development

The system development consists of two components which is the game environment and the MARQ framework. The game environment will be developed to serve as the interface for human players and the world in which the AI agent operates. While the MARQ framework provides the trained model to be used as the bot playing against the player.

4.2.1 Piece Value Computation

The reward function requires a quantitative assessment of piece value to guide learning. This section presents an analytical method for computing piece values based on probability theory, providing a deterministic alternative to empirical estimation through gameplay simulation.

Combat Dominance Score

The Combat Dominance Score (CDS) represents the probability that a given piece type defeats a uniformly random opponent piece. Let $\mathcal{P}$ denote the set of all piece types and n_i the count of piece type i in standard Stratego. The CDS for piece p is defined as:

$$\text{CDS}(p) = \frac{\sum_{i \in \mathcal{P}} \nVdash_{p \succ i} \cdot n_i}{N} \tag{4.1}$$

where $N = \sum_{i \in \mathcal{P}} n_i = 40$ is the total piece count, $\nVdash_{p \succ i}$ is an indicator function equal to 1 if piece p defeats piece i according to Stratego combat rules, and 0 otherwise. The combat rules specify that higher-ranked pieces defeat lower-ranked pieces, with exceptions: the Spy defeats the Marshal when attacking, and the Miner defeats Bombs.

Survival Rate

The Survival Rate (SR) measures the expected probability of a piece surviving an attack from a uniformly random attacker. Let $\mathcal{M} \subset \mathcal{P}$ denote the set of movable piece types (excluding Flag and Bomb). The SR for piece p is:

$$\mathrm{SR}(p) = \frac{\sum_{a \in \mathcal{M}} \nVdash_{p \succeq a} \cdot n_a}{\sum_{a \in \mathcal{M}} n_a} \tag{4.2}$$

where $\nVdash_{p \succeq a}$ equals 1 if piece p survives an attack from piece a (i.e., p defeats a or both are destroyed).

Auxiliary Value Components

Three additional factors contribute to piece value:

Strategic Ability Bonus. Certain pieces possess capabilities that extend beyond their combat rank. Table 4.1 enumerates these bonuses.

Table 4.1: Strategic ability bonuses for pieces with special capabilities.

Piece	Bonus	Description
Miner	2.0	Sole piece capable of removing Bombs
Spy	1.5	Defeats Marshal when initiating attack
Scout	1.0	Multi-square movement capability
Marshal	0.5	Highest combat rank

Mobility Factor. The Scout receives a mobility bonus of 1.0 due to its ability to move multiple squares per turn; all other movable pieces receive a base mobility of 0.5. Immovable pieces (Flag, Bomb) receive 0.

Scarcity Factor. The scarcity of each piece type influences its strategic importance. This is computed using logarithmic scaling:

$$\mathrm{SF}(p) = \log\left(\frac{\max_{i \in \mathcal{P}} n_i}{n_p}\right) + 1 \tag{4.3}$$

Composite Value Function

The final piece value is a weighted linear combination of the above components:

$$V(p) = \alpha_1 \cdot \mathrm{CDS}(p) + \alpha_2 \cdot \mathrm{SR}(p) + \alpha_3 \cdot S(p) + \alpha_4 \cdot M(p) + \alpha_5 \cdot \mathrm{SF}(p) \tag{4.4}$$

where $S(p)$ is the strategic ability bonus, $M(p)$ is the mobility factor, and weights $\alpha = (0.40, 0.25, 0.20, 0.10, 0.05)$ were selected to prioritize combat capability while accounting for strategic factors. Values are normalized such that $V(\text{Scout}) = 1.0$, analogous to the convention in chess where the Pawn serves as the unit of measurement.

Table 4.2: Computed piece values with associated combat and survival metrics.

Piece	**Value**	**CDS**	**SR**
Marshal	8.4	0.825	0.939
General	8.0	0.800	0.939
Colonel	7.5	0.750	0.879
Major	6.7	0.675	0.788
Captain	5.7	0.575	0.667
Lieutenant	4.8	0.475	0.545
Miner	4.3	0.400	0.273
Sergeant	3.8	0.375	0.424
Spy	1.1	0.050	0.000
Scout	1.0	0.050	0.030

These values inform the material reward component of the reward shaping function described in subsequent sections.

4.2.2 Handling Imperfect Information: Probabilistic Belief State (PBS)

A fundamental challenge in Stratego is the hidden identity of opponent pieces. To address this, the system incorporates a Probabilistic Belief State (PBS), represented as a matrix that maintains a probability distribution over the possible identities of each unrevealed opponent piece. The PBS is updated throughout the game through two complementary reasoning mechanisms.

Deductive reasoning applies when direct piece interactions occur. Upon combat, the identity of the involved pieces becomes known with certainty, and the PBS is updated accordingly. The probability mass previously assigned to eliminated possibilities is redistributed among remaining candidates through renormalization.

Inductive reasoning enables the system to infer piece identity from observed behavior patterns. Pieces exhibiting high mobility and long-range movement are assigned increased probability of being Scouts. Conversely, pieces that retreat from threats or remain stationary near the back rank suggest defensive behavior characteristic of high-value pieces such as the Flag or Marshal. These behavioral heuristics are incorporated through Bayesian updates to the belief distribution.

During training, the PBS is compared against the true game state to provide supervised learning signal. This comparison allows the belief estimation network to be trained toward more accurate predictions. The trained PBS effectively transforms the imperfect information game into an approximate perfect information representation, enabling the application of standard reinforcement

learning techniques.

4.3 Model Training

Training the MARQ agent is computationally intensive due to the large state-action space and the need to learn accurate belief state representations over extended game histories.

Rainbow DQN Architecture. MARQ employs a Rainbow DQN architecture that integrates several algorithmic improvements over the standard DQN. The implementation includes Categorical DQN (C51) for distributional value estimation with 51 atoms over the range $[V_{min}, V_{max}] = [-10, 10]$, Noisy Networks for learned exploration (replacing ϵ-greedy), and Dueling Network Architecture separating state value and action advantage estimation. The noisy linear layers inject state-dependent stochasticity, enabling the agent to learn appropriate exploration levels for different game situations without manual scheduling.

Network Structure. The Q-network processes a 27-channel state tensor through a convolutional encoder consisting of two layers (32 filters and 64 filters, both with 3×3 kernels and ReLU activation) followed by dueling streams. The state representation consists of 12 channels for PBS belief distributions (one per piece type), plus additional channels encoding the player's known pieces, board validity masks, and positional features. The AAREN module uses 24-dimensional action feature vectors, 64 hidden units, and 3 layers to process action histories and output piece type probability distributions.

Training Hyperparameters. The training configuration uses the following key parameters: learning rate $\alpha = 0.0001$ with Adam optimizer, discount factor $\gamma = 0.99$, batch size of 128 transitions, replay buffer capacity of 150,000 transitions, and target network synchronization every 5,000 steps. The model updates are performed every 32 steps to balance training stability with simulation throughput. The PBS is updated every 2 steps to balance inference accuracy with computational efficiency. Training proceeds for 35,000 episodes with model checkpoints saved every 250 episodes.

League-Based Training. To prevent policy cycling and ensure robustness, the agent trains against a diverse opponent pool. The opponent selection distribution allocates 50% to historical

league agents, 20% to random agents, 20% to greedy heuristic agents, and 10% to self-play. Agent checkpoints are added to the league every 500 episodes, with a maximum pool size of 50 historical agents.

Curriculum Learning. Training follows a five-phase curriculum of increasing complexity. Phase 1 (Full Observability) allows the agent to see all piece identities, learning basic game mechanics against random and heuristic opponents for 500–2,000 episodes until achieving $\geq$90% win rate against random. Phase 2 (Partial Observability) disables full observability, requiring reliance on PBS predictions for 1,000–3,000 episodes until PBS accuracy reaches $\geq$70%. Phase 3 (Self-Play) trains against a delayed copy of the agent for 2,000–5,000 episodes to discover novel strategies. Phase 4 (League Training) activates the full opponent distribution for 5,000+ episodes. Phase 5 (Scenario Drills) introduces specialized board configurations every 1,000 episodes for targeted skill training.

Reward Shaping Weights. The composite reward function uses configurable weights: outcome reward $w_o = 1.0$, material reward $w_m = 0.5$, epistemic reward $w_e = 0.3$, and positional reward $w_p = 0.2$. These weights prioritize terminal outcomes while providing dense learning signals through intermediate rewards.

Distributional RL-Compatible Reward Normalization. The C51 distributional architecture requires careful reward scaling to ensure cumulative returns fall within the value distribution support $[V_{min}, V_{max}]$. With a support of $[-3, +3]$ and 51 atoms, each atom spans 0.12 value units, providing sufficient resolution for learning the return distribution.

To address reward hacking behavior—where the agent learns to stall games and force draws due to risk aversion toward unknown enemy piece ranks—the framework implements an anti-stall reward structure with normalized magnitudes. Terminal rewards are set to $+1.0$ for wins and -1.0 for losses, well within the distribution bounds. Draws and timeouts receive -0.8, deliberately *worse* than a fighting loss, which creates a gradient toward aggressive play.

A constant time-pressure penalty of -0.005 per step creates implicit preference for shorter games. Over a typical 100-step game, this accumulates to approximately -0.5, keeping cumulative returns within bounds.

Combat rewards provide material signal through capture bonus $+0.1 \cdot (r_{enemy}/10)$, where r_{enemy} is the captured piece's rank, yielding a maximum of $+0.1$ per capture. Losing a piece incurs -0.05,

which is less severe than passive play to encourage probing attacks.

A crucial component for distributional learning is the information gain bonus. Revealing an enemy piece rank through combat yields +0.02, with an additional +0.03 for the first reveal of each piece type. This helps the value distribution learn the *variance* associated with attacking unknown pieces—unrevealed pieces have high uncertainty, while revealed pieces have low variance. The distributional approach naturally captures this risk-reward tradeoff that scalar Q-learning cannot represent.

Strategic capture bonuses encode domain knowledge: eliminating the opponent's Spy while their Marshal survives grants +0.15, and removing enemy Miners while Bombs remain yields +0.08.

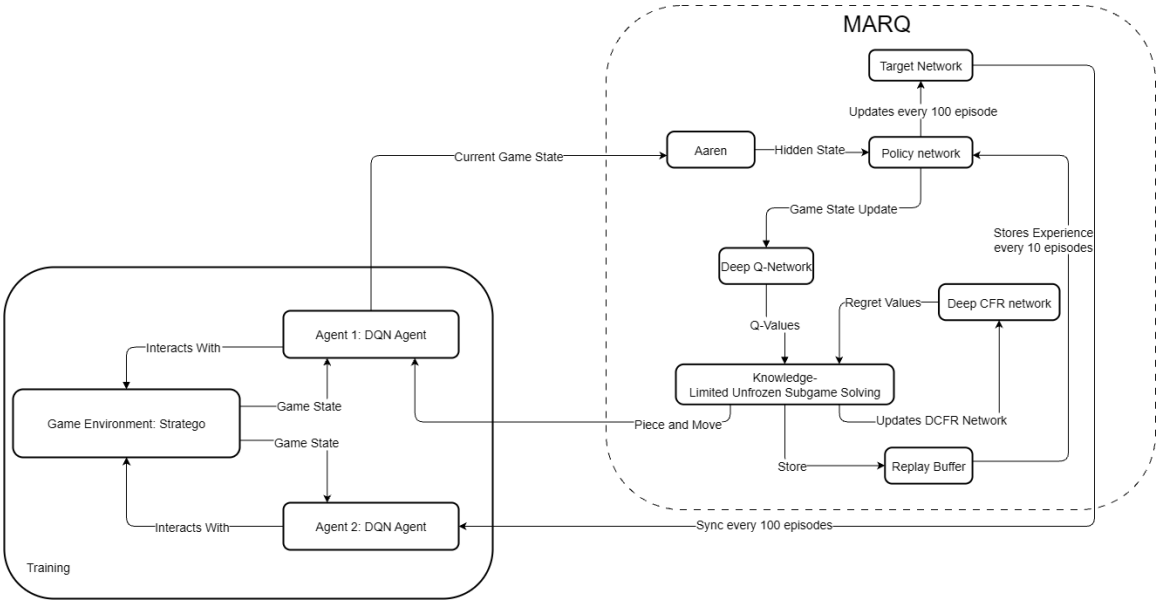

Figure 4.1: The MARQ System Architecture, which takes the Board State and PBS as input.

4.3.1 Deployment

Upon completion of the training and validation cycles, the best-performing MARQ model weights will be saved and finalized for deployment. Deploy the trained model into the Stratego game environment described in Section 2 of the methodology.

The deployed system will function as the complete application used during the user testing phase. It will be configured to run locally on the single device designated for the experimental procedures.

In this production mode, the model's exploratory functions will be deactivated for a more measured approach from the agent, always selecting the action with the highest predicted Q-value. This ensures that all human participants compete against the identical, optimized version of the MARQ framework, providing a consistent baseline for the quantitative and qualitative evaluations.

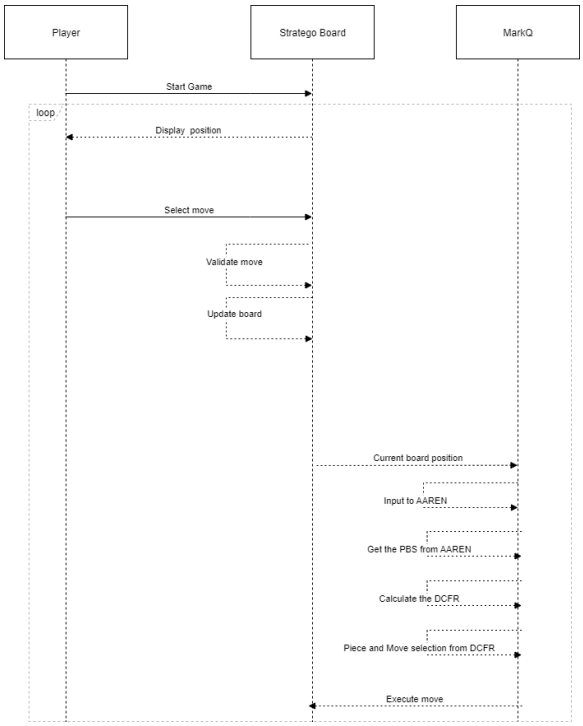

Figure 4.2: Game Sequence Diagram

In the figure above, the player starts the game and the loop begins until either the player wins by capturing the opponent's flag or loses if the agent captures the player's flag. The agent receives the current board state and updates the AAREN module, which constructs the PBS representing the probability distribution over hidden piece identities. The DQN then evaluates Q-values for all legal actions given this belief-enhanced state representation and selects the action with the highest expected value.

4.4 Evaluation

A comprehensive evaluation approach combining quantitative and qualitative methods will be used to assess the MARQ framework's performance and the user experience.

Quantitative Analysis. The quantitative assessment of the model's performance tracks several key metrics to gauge learning progress and strategic capability. Win rate percentage measures the proportion of games won against human participants. Average game length, computed as the mean number of moves per game, indicates decision efficiency. Decision-making speed records the average time taken by the agent to select an action. Training stability is assessed through loss curves showing the progression of distributional and value losses over training episodes. To evaluate the synchronized learning environment, a two-tailed paired t-test will determine if there is a statistically significant difference in gameplay metrics between a player's first and last games, assessing whether human players improve against the AI. The results from Likert-scale questions will be analyzed using a weighted mean to derive a quantitative score for each dimension of user acceptance.

Qualitative Analysis. To assess end-user opinion on usability, a usability testing session will gather participant feedback through an evaluation form evaluating three key dimensions. Performance assessment examines the agent's speed, consistency, and perceived strategic competence. Usability evaluation considers the ease of use, intuitiveness of the interface, and overall user experience. Relevance gauges the AI's value as a strategic training tool and its potential for helping players learn Stratego.

Model Iteration and Initial Results. The development of the MARQ framework is an iterative process where each model version represents a snapshot of the system's evolution. Appendix B presents initial training results from 1,000 episodes. During this early phase, the agent remains in an exploratory state, searching for exploitable strategies. The reward signal indicates difficulty in discovering effective tactics against the opponent. Early win rates reflect frequent draws caused by repetitive piece movements. With the transition to Noisy Networks, the agent's stochasticity is learned rather than scheduled, allowing more adaptive exploration throughout training.

Appendix A

Gantt Chart

This section includes the images of the Gantt chart that will represent the timeline for the whole duration of this study.

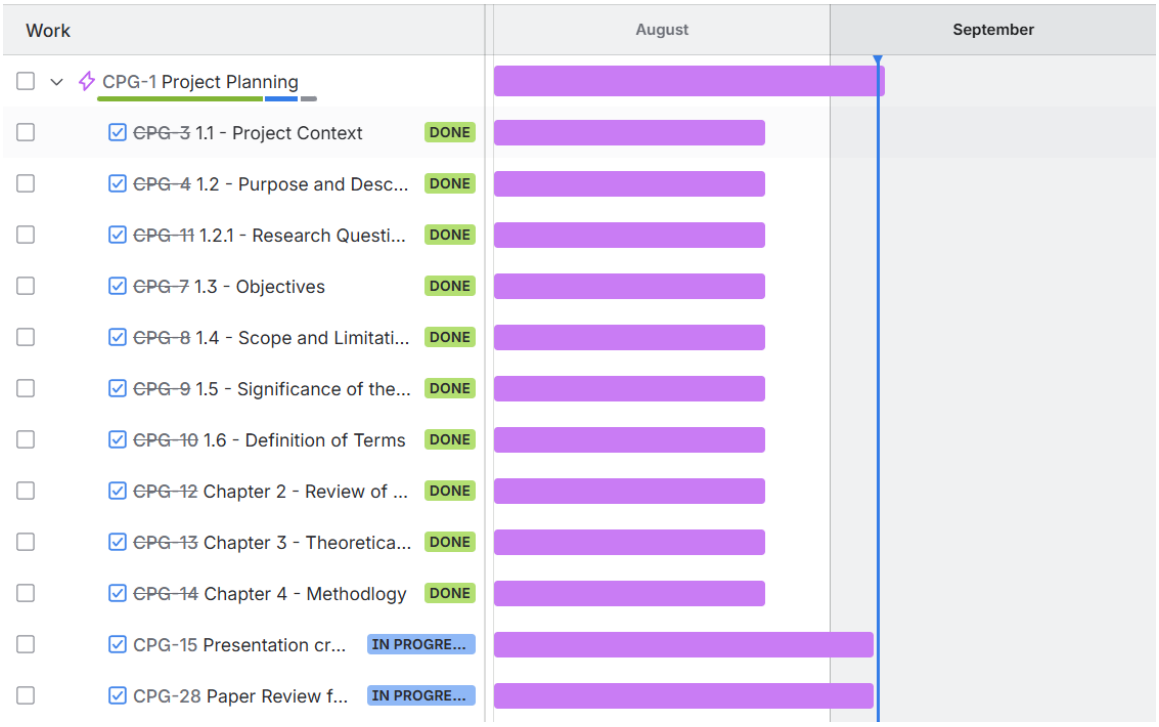

Figure A.1: Timeline: August 1 - September 5

Figure A.2: Timeline: September 11 - November 30

Figure A.3: Timeline: October 27 - January 27

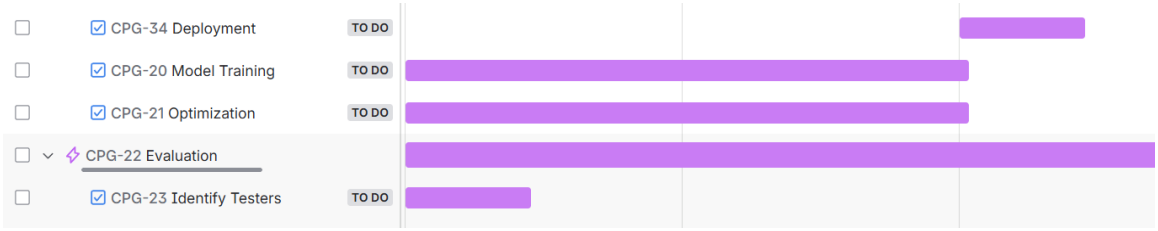

Figure A.4: Timeline: December 1 - February 14

Figure A.5: Timeline: February 15 - May 4

Appendix B

System UI and Training

This section includes the images of the system UI and Training.

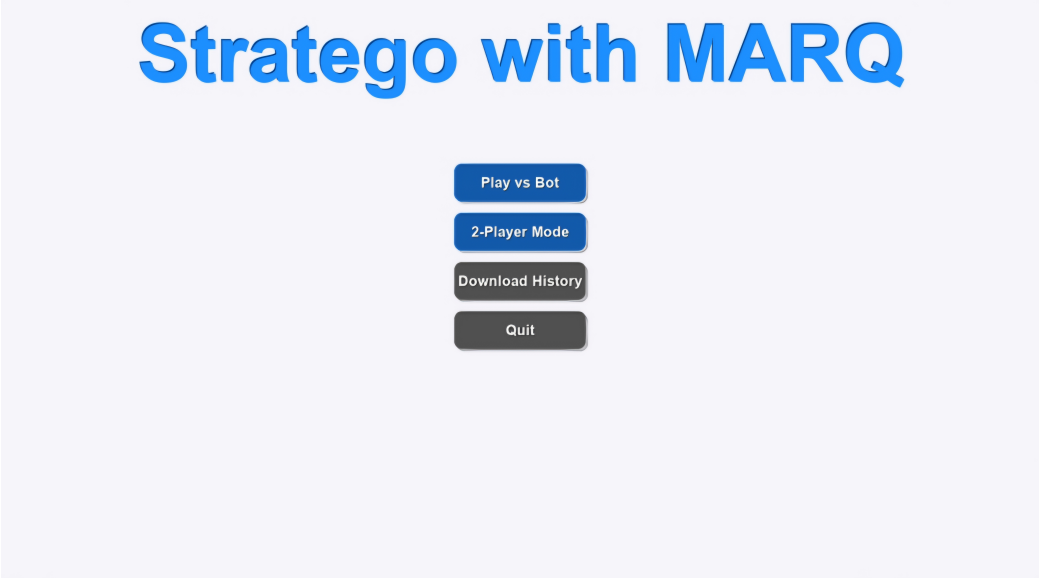

Figure B.1: Game Menu

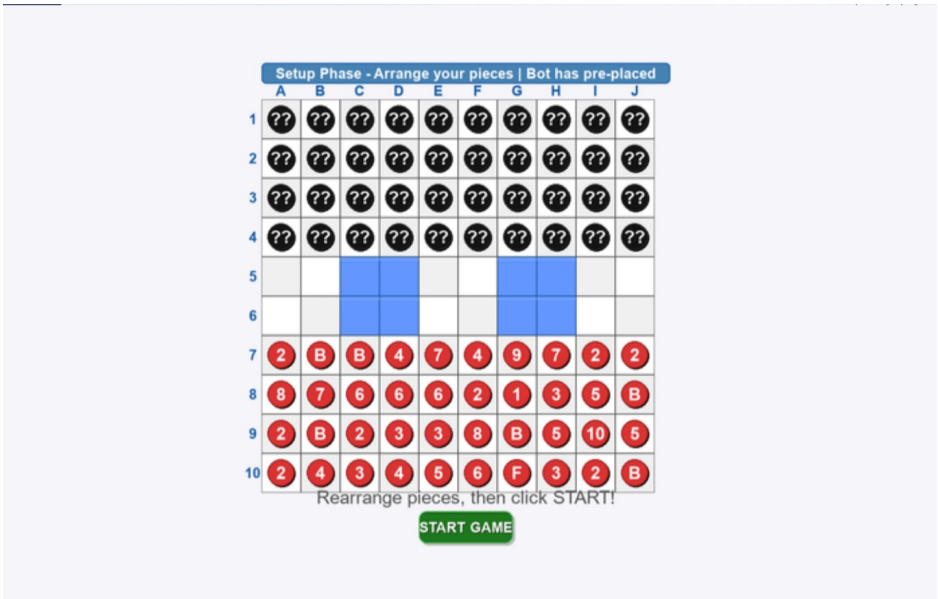

Figure B.2: Setup Phase

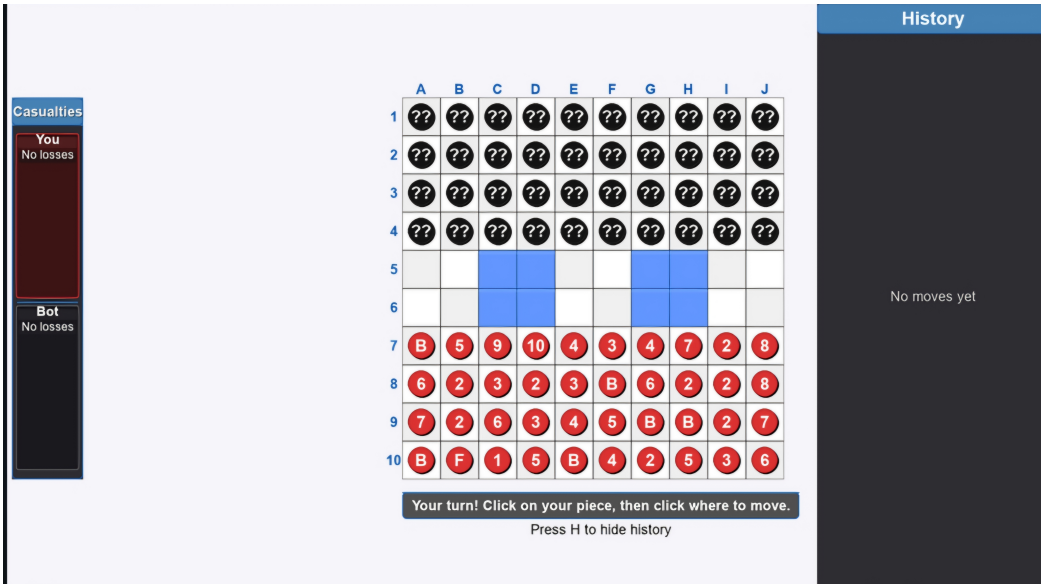

Figure B.3: Game Start

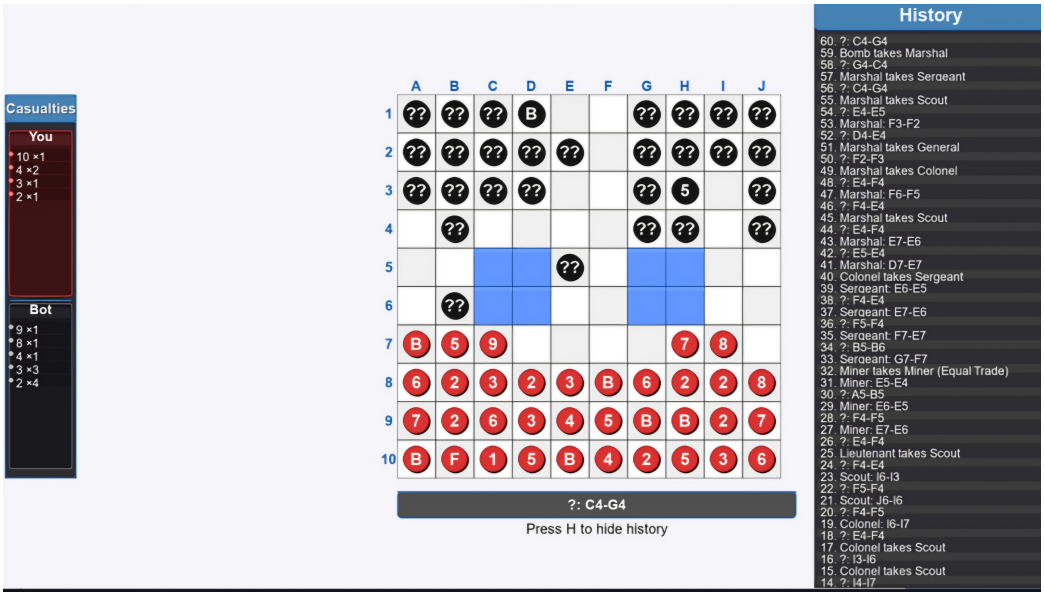

Figure B.4: Battle and History

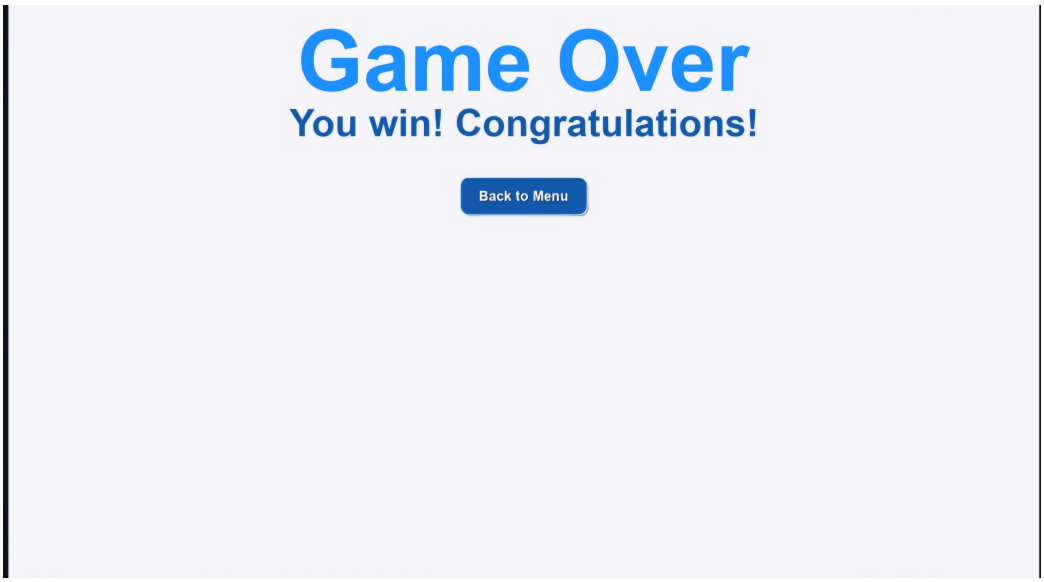

Figure B.5: Victory Screen

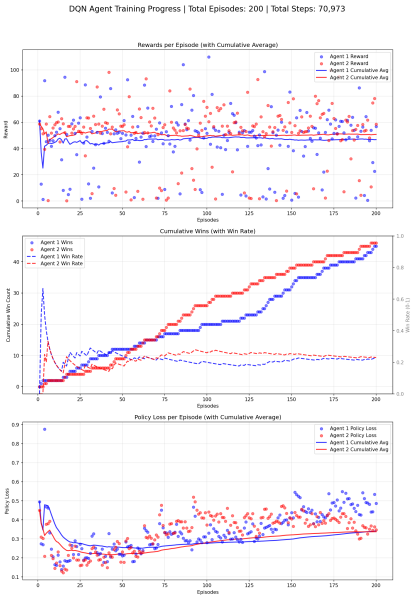

Figure B.6: Training at 200 Episodes

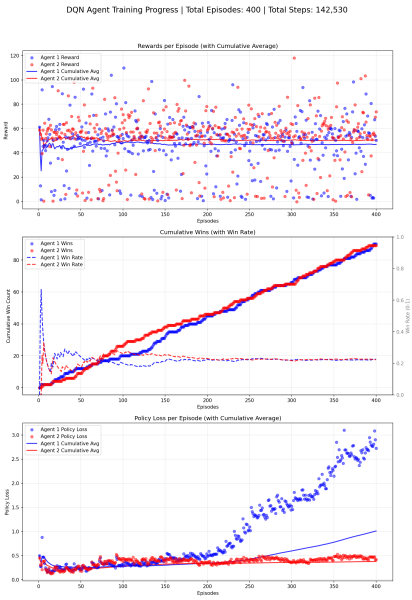

Figure B.7: Training at 400 Episodes

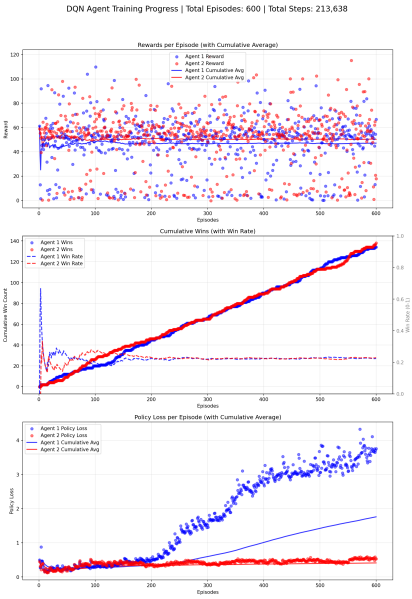

Figure B.8: Training at 600 Episodes

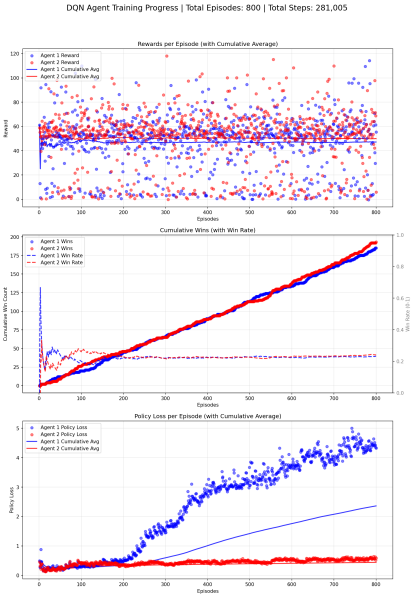

Figure B.9: Training at 800 Episodes

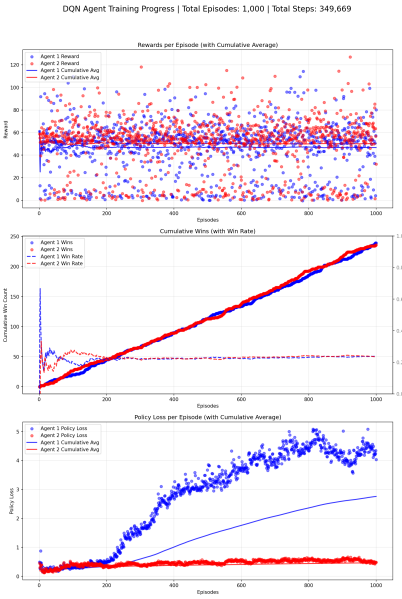

Figure B.10: Training at 1000 Episodes

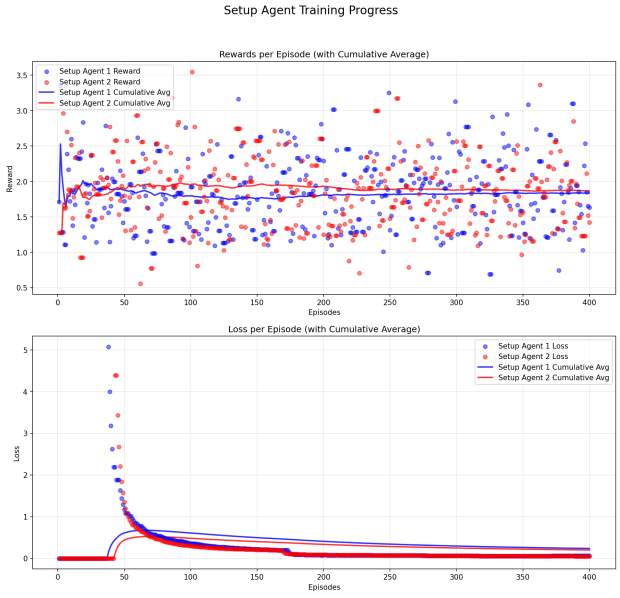

Figure B.11: Setup Agent Training

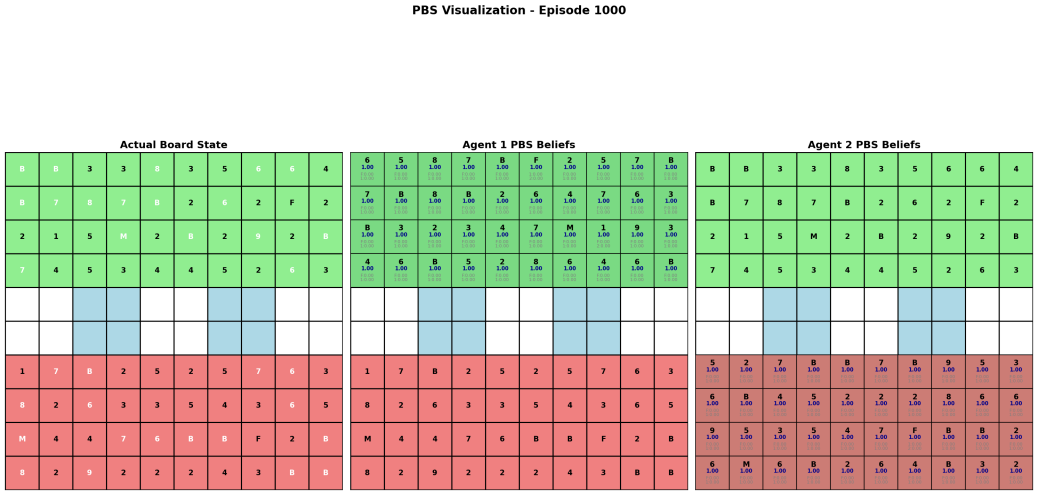

Figure B.12: PBS Visualization

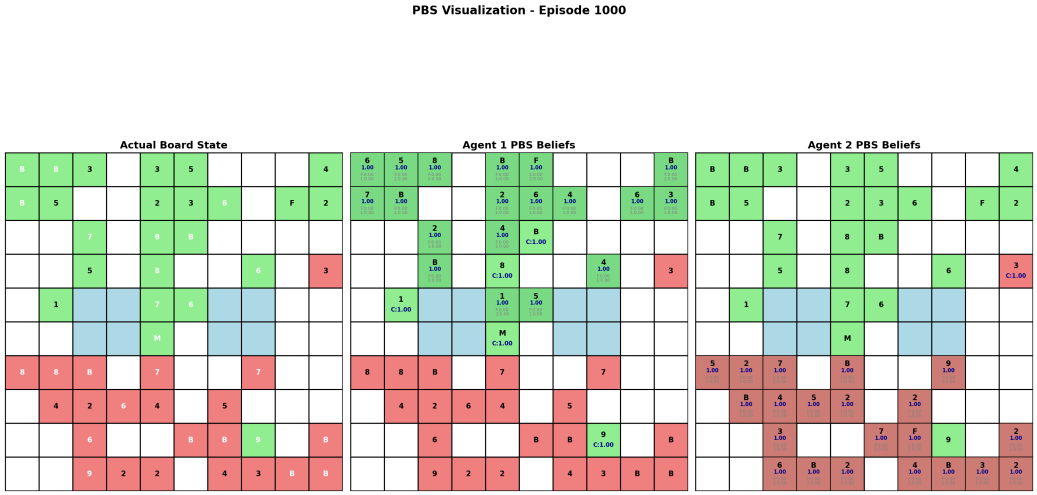

Figure B.13: PBS Visualization

REFERENCES

[1] G. Adams, S. Padmanabhan, and S. Shekhar, *Resolving implicit coordination in multi-agent deep rl with deep q-networks & game theory*, arXiv preprint arXiv:2012.09136, (2023).

[2] S. M. Al-Selwi, M. F. Hassan, S. J. Abdulkadir, A. Muneer, E. H. Sumiea, A. Alqushaibi, and M. G. Ragab, *Rnn-lstm: From applications to modeling techniques and beyond—systematic review*, Journal of King Saud University - Computer and Information Sciences, 36 (2024), p. 102068.

[3] R. N. Amaliani, D. W. Soewardikoen, H. Azhar, and Y. Rahman, *Designing board games as an enhancer of a sense of community and cognition for the elderly of tresna werdha budi pertiwi social care*, Advances in Social Humanities Research, 3 (2025).

[4] G. Barzilai, O. Shamir, and M. Zamani, *Are convex optimization curves convex?*, arXiv preprint arXiv:2503.10138, (2025).

[5] T. Becker and Z. Sunberg, *Imperfect information games and counterfactual regret minimization in space domain awareness*, in Advanced Maui Optical and Space Surveillance Technologies Conference (AMOS), AMOS Technologies, 2022.

[6] A. Brim, *Deep reinforcement learning pairs trading with a double deep q-network*, in Proceedings of the 10th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, Nevada, USA, Jan.–2020 2020, pp. 222–227.

[7] N. Brown, A. Lerer, S. Gross, and T. Sandholm, *Deep counterfactual regret minimization*, in International conference on machine learning, PMLR, 2019, pp. 793–802.

[8] L. Canese, G. C. Cardarilli, L. Di Nunzio, R. Fazzolari, D. Giardino, M. Re, and S. Spanò, *Multi-agent reinforcement learning: A review of challenges and applications*, Applied Sciences, 11 (2021), p. 4948. Article number; MDPI journal uses article numbers instead of page ranges.

[9] P. P. Chitayat, A. M. Colman, R. J. Hyde, A. Drachen, and P. Cowling, *Wards: Modelling the worth of vision in MOBA's*, in Intelligent Systems and Applications - Proceedings of the 2020 Intelligent Systems Conference (IntelliSys), vol. 1251 of Advances in Intelligent Systems and Computing, Springer, 2020, pp. 55–76.

[10] H. M. Dbouk, K. Ghorayeb, H. Kassem, H. Hayek, R. Torrens, and O. Wells, *Facility placement layout optimization*, Journal of Petroleum Science and Engineering, 207 (2021), p. 109079.

[11] B. De Schutter, R. Babuska, and L. Busoniu, *A comprehensive survey of multi-agent reinforcement learning*, IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews, 38 (2008), pp. 156–172.

[12] I. El Shar and D. Jiang, *Weakly coupled deep q-networks*, in Advances in Neural Information Processing Systems 36, Curran Associates, Inc., 2023. NeurIPS 2023 Conference Track.

[13] L. Feng, F. Tung, H. Hajimirsadeghi, M. O. Ahmed, Y. Bengio, and G. Mori, *Attention as an rnn*, arXiv preprint arXiv:2405.13956, (2024).

[14] J. Harrington, *Let's meetup? board game communities in hong kong*, Games and Culture, 20 (2025), pp. 279–297.

[15] C. Hu, Y. Zhao, Z. Wang, H. Du, and J. Liu, *Games for artificial intelligence research: A review and perspectives*, IEEE Transactions on Artificial Intelligence, 5 (2024), pp. 5949–5968.

[16] Y. Huang, *Deep q-networks*, in Deep Reinforcement Learning: Fundamentals, Research, and Applications, S. Z. Hao Dong, Zihan Ding, ed., Springer Nature, 2020, ch. 4, pp. 135–160. `http://www.deepreinforcementlearningbook.org`.

[17] Y. Huang, *Deep Q-Networks*, Springer Singapore, Singapore, 2020, pp. 135–160.

[18] D. Hugi and E. Izy, *Multi-agent deep reinforcement learning (marl) in starcraft 2.* `https://crml.eelabs.technion.ac.il/projects/multi-agent-deep-reinforcement-learning-marl-in-starcraft-2/`, 2017. Course project, Winter semester.

[19] A. Kaur, K. Kumar, A. Prakash, and R. Tripathi, *Imperfect csi-based resource management in cognitive iot networks: A deep recurrent reinforcement learning framework*, IEEE Transactions on Cognitive Communications and Networking, 9 (2023), pp. 1271–1281.

[20] A. Kensert, P. Libin, G. Desmet, and D. Cabooter, *Deep reinforcement learning for the direct optimization of gradient separations in liquid chromatography*, Journal of Chromatography A, 1720 (2024), p. 464768.

[21] P. Koudela, *Game of incomplete information and national security*, Central European Political Science Review (CEPSR), 20 (2019), pp. 73–96.

[22] B. Li, Z. Fang, and L. Huang, *Rl-cfr: Improving action abstraction for imperfect information extensive-form games with reinforcement learning*, arXiv preprint arXiv:2403.04344, (2024).

[23] S. Li, Y. Zhang, X. Wang, W. Xue, and B. An, *Cfr-mix: Solving imperfect information extensive-form games with combinatorial action space*, in Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21), IJCAI, 2021, pp. 3663–3669.

[24] I. Loshchilov and F. Hutter, *Decoupled weight decay regularization*, arXiv preprint arXiv:1711.05101, (2017).

[25] Z. Luo, Z. Chen, and J. Welsh, *Multi-agent reinforcement learning with deep networks for diverse q-vectors*, arXiv preprint arXiv:2406.07848v1, (2024). Version v1, June 2024.

[26] R. R. Nair, T. Babu, S. Kishore, K. Pavithra, and S. G. Sumana, *Improving alpha-beta pruning efficiency in chess engines*, in 2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD), Manama, Bahrain, 2025, IEEE, pp. 1–6.

[27] X. Ouyang and T. Zhou, *Imperfect-information game ai agent based on reinforcement learning using tree search and a deep neural network*, Electronics, 12 (2023), p. 2453.

[28] J. Perolat, B. De Vylder, D. Hennes, E. Tarassov, F. Strub, V. de Boer, P. Muller, J. T. Connor, N. Burch, T. Anthony, et al., *Mastering the game of stratego with model-free multiagent reinforcement learning*, Science, 378 (2022), pp. 990–996.

[29] C. Petersen, *The reality of the board game community: And the connection, identity and experience that creates it*, master's thesis, Eastern University, Aug. 2024.

[30] T. Qu, Q. Zhou, J. Zhu, and F. Duan, *Strategy optimization of imperfect information games based on nfsp with ddqn*, in Advances in Guidance, Navigation and Control, L. Yan, H. Duan, and Y. Deng, eds., vol. 845 of Lecture Notes in Electrical Engineering, Springer, Singapore, 2023, pp. 4376–4383.

[31] A. Saffidine, H. Finnsson, and M. Buro, *Alpha-beta pruning for games with simultaneous moves*, in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 26, 2021, pp. 556–562.

[32] M. Schmid, M. Moravčík, N. Burch, R. Kadlec, J. Davidson, K. Waugh, N. Bard, F. Timbers, M. Lanctot, G. Z. Holland, et al., *Student of games: A unified learning algorithm for both perfect and imperfect information games*, Science Advances, 9 (2023), p. eadg3256.

[33] M. Schmid, M. Moravčík, N. Burch, R. Kadlec, J. Davidson, K. Waugh, N. Bard, F. Timbers, M. Lanctot, G. Z. Holland, E. Davoodi, A. Christianson, and M. Bowling, *Student of games: A unified learning algorithm for both perfect and imperfect information games*, Science Advances, 9 (2023), p. eadg3256.

[34] M. Schofield and M. Thielscher, *General game playing with imperfect information*, Journal of Artificial Intelligence Research, 66 (2019), pp. 901–935.

[35] H. Xu, K. Li, H. Fu, Q. Fu, and J. Xing, *Autocfr: Learning to design counterfactual regret minimization algorithms*, in Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22), Association for the Advancement of Artificial Intelligence, 2022.

[36] H. Xu, K. Li, H. Fu, Q. FU, J. Xing, and J. Cheng, *Dynamic discounted counterfactual regret minimization*, in The Twelfth International Conference on Learning Representations, 2024.

[37] A. Zakharenkov and I. Makarov, *Deep reinforcement learning with dqn vs. ppo in vizdoom*, in 2021 IEEE 21st International Symposium on Computational Intelligence and Informatics (CINTI), Budapest, Hungary, 2021, IEEE, pp. 131–136.

[38] B. Zhang and T. Sandholm, *Subgame solving without common knowledge*, Advances in Neural Information Processing Systems, 34 (2021), pp. 23993–24004.

[39] B. H. Zhang and T. Sandholm, *General search techniques without common knowledge for imperfect-information games, and application to superhuman fog of war chess*, arXiv preprint arXiv:2506.01242, (2025).

[40] K. Zhang, Z. Yang, and T. Başar, *Multi-agent reinforcement learning: A selective overview of theories and algorithms*, Handbook of reinforcement learning and control, (2021), pp. 321–384.

[41] S. Zheng, A. Trott, S. Srinivasa, N. Naik, M. Gruesbeck, D. C. Parkes, and R. Socher, *The ai economist: Improving equality and productivity with ai-driven tax policies*, arXiv preprint arXiv:2004.13332, (2020).

[42] K. Zhu, Q. Liu, X. Yan, F. Wu, X. Zhao, P. Zhou, and X. Yang, *A comprehensive survey of multi-agent reinforcement learning*, IEEE Transactions on Neural Networks and Learning Systems, 34 (2022), pp. 3074–3093.

VITA

James Gabriel P. Mabagos is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.

Mark Lawrence M. Quibot is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.